

HRMS Project

Project Overview:

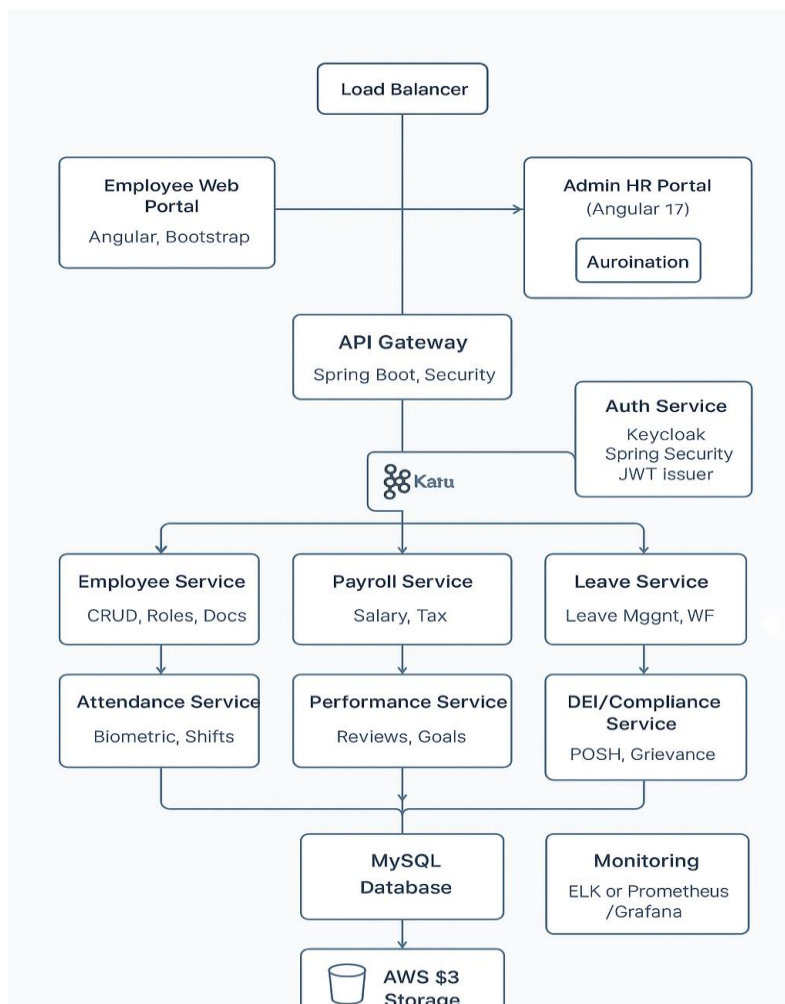
This document contains the high-level system architecture diagram for the HRMS Application. The diagram shows the interaction between Employee Portal, Admin HR Portal, API Gateway, Microservices, Database, and AWS Storage.

Flow/Path: -

Employee → Submit Details → HR Verifies → Auto ID/Email Generated →

Docs Uploaded to S3 → Manager Assigned → Portal Access Granted

Flow Diagram: -



Spring Boot Project Setup: -

- Create new project in IntelliJ (Maven).
- Added some Dependencies in pom.xml:
 - Spring Web (REST API)
 - Spring Data JPA (DB integration)
 - Spring Security (JWT)
 - Spring Validation
 - MySQL Connector
 - AWS SDK (for S3 file storage)
 - Lombok (optional, less boilerplate)

Module 1.1 Dashboard Admin/ Employee (Shraddha Bobe)

Key Entities and Attribute: -

1.01 Admin Dashboard: -

Employee Table Structure: -

Column Name	Data Type	Constraints
EmployeeID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
FirstName	VARCHAR (50)	NOT NULL
LastName	VARCHAR (50)	NOT NULL
Gender	ENUM('Male','Female','Other')	NOT NULL
DateOfBirth	DATE	NOT NULL
Phone	VARCHAR (15)	UNIQUE
Email	VARCHAR (100)	UNIQUE
Address	VARCHAR (255)	NULL
DepartmentID	INT	FOREIGN KEY REFERENCES Department (DepartmentID)
RoleDesignation	VARCHAR (100)	NOT NULL
DateOfJoining	DATE	NOT NULL
EmploymentType	ENUM('Permanent','Contract','Intern')	NOT NULL
Status	ENUM('Active','Resigned','Retired')	NOT NULL
ManagerID	INT	NULL (self-referencing FK to EmployeeID)
ProbationEndDate	DATE	NULL
WorkLocation	ENUM('Onsite','Remote','Hybrid')	NOT NULL

```
CREATE TABLE Employee ( EmployeeID INT PRIMARY KEY AUTO_INCREMENT,
  FirstName VARCHAR (50) NOT NULL,
  LastName VARCHAR (50) NOT NULL,
```

Gender ENUM('Male','Female','Other') NOT NULL,
 DateOfBirth DATE NOT NULL,
 Phone VARCHAR (15) UNIQUE,
 Email VARCHAR (100) UNIQUE,
 Address VARCHAR (255),
 DepartmentID INT,
 RoleDesignation VARCHAR (100) NOT NULL,
 DateOfJoining DATE NOT NULL,
 EmploymentType ENUM('Permanent','Contract','Intern') NOT NULL,
 Status ENUM('Active','Resigned','Retired') NOT NULL,
 ManagerID INT,
 ProbationEndDate DATE,
 WorkLocation ENUM('Onsite','Remote','Hybrid') NOT NULL,
 FOREIGN KEY (DepartmentID) REFERENCES Department (DepartmentID),
 FOREIGN KEY (ManagerID) REFERENCES Employee (EmployeeID));

Department Table Structure: -

Column Name	Data Type	Constraints
DepartmentID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
DepartmentName	VARCHAR (100)	NOT NULL
HeadOfDepartment	INT	FOREIGN KEY REFERENCES Employee (EmployeeID)
BudgetAllocation	DECIMAL (12,2)	NOT NULL

```

CREATE TABLE Department (
  DepartmentID INT PRIMARY KEY AUTO_INCREMENT,
  DepartmentName VARCHAR (100) NOT NULL,
  HeadOfDepartment INT,
  BudgetAllocation DECIMAL (12,2) NOT NULL,
  FOREIGN KEY (HeadOfDepartment) REFERENCES Employee (EmployeeID)
);
  
```

Attendance Table Structure: -

Column Name	Data Type	Constraints
AttendanceID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
EmployeeID	INT	FOREIGN KEY REFERENCES Employee (EmployeeID)
Date	DATE	NOT NULL
Status	ENUM	ENUM ('Present', 'Absent', 'On Leave', 'Remote')
CheckInTime	TIME	NULL (if absent/leave, may remain NULL)
CheckOutTime	TIME	NULL (if absent/leave, may remain NULL)

```
CREATE TABLE Attendance ( AttendanceID INT PRIMARY KEY AUTO_INCREMENT,
EmployeeID INT NOT NULL,
Date DATE NOT NULL, Status ENUM ('Present', 'Absent', 'On Leave', 'Remote') NOT NULL,
CheckInTime TIME,
CheckOutTime TIME,
FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID) )
```

Leave Table Structure: -

Column Name	Data Type	Constraints
LeaveID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
EmployeeID	INT	FOREIGN KEY REFERENCES Employee (EmployeeID) NOT NULL
LeaveType	ENUM ('Casual', 'Sick', 'Paid', 'Unpaid', 'Maternity', 'Paternity')	NOT NULL
StartDate	DATE	NOT NULL
EndDate	DATE	NOT NULL
Status	ENUM ('Applied', 'Approved', 'Rejected')	NOT NULL
ApproverID	INT	FOREIGN KEY REFERENCES Employee (EmployeeID), NULL allowed

```
CREATE TABLE LeaveRecords ( LeaveID INT PRIMARY KEY AUTO_INCREMENT,
EmployeeID INT NOT NULL,
LeaveType ENUM ('Casual', 'Sick', 'Paid', 'Unpaid', 'Maternity', 'Paternity') NOT NULL,
StartDate DATE NOT NULL,
EndDate DATE NOT NULL,
Status ENUM ('Applied', 'Approved', 'Rejected') NOT NULL,
ApproverID INT, FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID),
FOREIGN KEY (ApproverID) REFERENCES Employee (EmployeeID) );
```

Recruitment/position Table Structure: -

Column Name	Data Type	Constraints
PositionID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
JobTitle	VARCHAR (100)	NOT NULL
DepartmentID	INT	FOREIGN KEY REFERENCES Department (DepartmentID) NOT NULL
OpeningsCount	INT	NOT NULL, CHECK (OpeningsCount >= 0)
Status	ENUM ('Open', 'In Progress', 'Closed')	NOT NULL

CreationDate	DATE	NOT NULL
HiringManagerID	INT	FOREIGN KEY REFERENCES Employee (EmployeeID) NOT NULL

```
CREATE TABLE Position (
  PositionID INT PRIMARY KEY AUTO_INCREMENT,
  JobTitle VARCHAR (100) NOT NULL,
  DepartmentID INT NOT NULL,
  OpeningsCount INT NOT NULL CHECK (OpeningsCount >= 0),
  Status ENUM ('Open', 'In Progress', 'Closed') NOT NULL,
  CreationDate DATE NOT NULL,
  HiringManagerID INT NOT NULL,
  FOREIGN KEY (DepartmentID) REFERENCES Department (DepartmentID),
  FOREIGN KEY (HiringManagerID) REFERENCES Employee (EmployeeID) );
```

Policy Table Structure: -

Column Name	Data Type	Constraints
PolicyID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
PolicyTitle	VARCHAR (150)	NOT NULL
Description	TEXT	NULL
EffectiveDate	DATE	NOT NULL
LastUpdated	DATE	NULL
Status	ENUM ('Active', 'Expired')	NOT NULL

```
CREATE TABLE Policy (
  PolicyID INT PRIMARY KEY AUTO_INCREMENT,
  PolicyTitle VARCHAR (150) NOT NULL,
  Description TEXT,
  EffectiveDate DATE NOT NULL,
  LastUpdated DATE,
  Status ENUM('Active','Expired') NOT NULL
);
```

PolicyAcknowledgment Table Structure: -

Column Name	Data Type	Constraints
AcknowledgmentID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
EmployeeID	INT	FOREIGN KEY REFERENCES Employee (EmployeeID) NOT NULL
PolicyID	INT	FOREIGN KEY REFERENCES Policy (PolicyID) NOT NULL

AcknowledgmentDate	DATE	NOT NULL
Status	ENUM ('Read', 'Unread')	NOT NULL

```
CREATE TABLE PolicyAcknowledgment ( AcknowledgmentID INT PRIMARY KEY AUTO_INCREMENT,
EmployeeID INT NOT NULL,
PolicyID INT NOT NULL, AcknowledgmentDate DATE NOT NULL,
Status ENUM('Read','Unread') NOT NULL,
FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID),
FOREIGN KEY (PolicyID) REFERENCES Policy (PolicyID) );
```

Policy Table Structure: -

Column Name	Data Type	Constraints
TrainingID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
TrainingName	VARCHAR (150)	NOT NULL
Description	TEXT	NULL
StartDate	DATE	NOT NULL
EndDate	DATE	NOT NULL
AssignedBy	INT	FOREIGN KEY REFERENCES Employee (EmployeeID)
		NOT NULL

```
CREATE TABLE Training ( TrainingID INT PRIMARY KEY AUTO_INCREMENT,
TrainingName VARCHAR (150) NOT NULL,
Description TEXT, StartDate DATE NOT NULL,
EndDate DATE NOT NULL, AssignedBy INT NOT NULL,
FOREIGN KEY (AssignedBy) REFERENCES Employee (EmployeeID) );
```

Training Completion Table Structure: -

Column Name	Data Type	Constraints
TrainingCompletionID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
EmployeeID	INT	FOREIGN KEY REFERENCES Employee (EmployeeID) NOT NULL

TrainingID	INT	FOREIGN KEY REFERENCES Training (TrainingID) NOT NULL
CompletionStatus	ENUM ('Not Started','In Progress','Completed')	NOT NULL
CompletionDate	DATE	NULL (only filled if status is Completed)

```
CREATE TABLE TrainingCompletion ( TrainingCompletionID INT PRIMARY KEY AUTO_INCREMENT,
EmployeeID INT NOT NULL,
TrainingID INT NOT NULL CompletionStatus ENUM ('Not Started','In Progress','Completed') NOT NULL,
CompletionDate DATE,
FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID),

FOREIGN KEY (TrainingID) REFERENCES Training (TrainingID) );
```

Notification Table Structure: -

Column Name	Data Type	Constraints
NotificationID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
Type	ENUM('Birthday','Compliance','Announcement','Payroll','General')	NOT NULL
Message	TEXT	NOT NULL
Priority	ENUM('High','Medium','Low')	NOT NULL
RelatedEntity	VARCHAR (50)	NULL (could store PolicyID, EmployeeID, etc.)
CreatedDate	DATETIME	NOT NULL
IsRead	ENUM('Yes','No')	NOT NULL DEFAULT 'No'

```
CREATE TABLE Notification ( NotificationID INT PRIMARY KEY AUTO_INCREMENT,
Type ENUM('Birthday','Compliance','Announcement','Payroll','General') NOT NULL,
Message TEXT NOT NULL,
Priority ENUM('High','Medium','Low') NOT NULL,
RelatedEntity VARCHAR (50),
CreatedDate DATETIME NOT NULL,
IsRead ENUM('Yes','No') NOT NULL DEFAULT 'No' )
```

Approval Table Structure: -

Column Name	Data Type	Constraints
ApprovalID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT

Module	ENUM('Leave','Timesheet','Payroll','Reimbursement','Training')	NOT NULL
RequestID	INT	Required, maps to the primary key of the respective module (external FK relationship handled logically)
ApproverID	INT	FOREIGN KEY REFERENCES Employee (EmployeeID) NOT NULL
Status	ENUM('Pending','Approved','Rejected')	NOT NULL
ActionDate	DATETIME	NULL (filled only when action taken)

```
CREATE TABLE Approval ( ApprovalID INT PRIMARY KEY AUTO_INCREMENT,
Module ENUM('Leave','Timesheet','Payroll','Reimbursement','Training') NOT NULL,
RequestID INT NOT NULL, ApproverID INT NOT NULL,
Status ENUM('Pending','Approved','Rejected') NOT NULL,
ActionDate DATETIME,
FOREIGN KEY (ApproverID) REFERENCES Employee (EmployeeID) );
```

Payroll Table Structure: -

Column Name	Data Type	Constraints
PayrollID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
EmployeeID	INT	FOREIGN KEY REFERENCES Employee (EmployeeID) NOT NULL
Month	TINYINT	NOT NULL, CHECK (Month BETWEEN 1 AND 12)
Year	YEAR	NOT NULL
GrossSalary	DECIMAL (12,2)	NOT NULL
NetSalary	DECIMAL (12,2)	NOT NULL
Deductions	DECIMAL (12,2)	NOT NULL DEFAULT 0.00
Status	ENUM('Pending','Processed')	NOT NULL
ProcessedDate	DATETIME	NULL (filled only once processed)

```
CREATE TABLE Payroll ( PayrollID INT PRIMARY KEY AUTO_INCREMENT,
EmployeeID INT NOT NULL,
Month TINYINT NOT NULL CHECK (Month BETWEEN 1 AND 12),
Year YEAR NOT NULL,
GrossSalary DECIMAL (12,2) NOT NULL,
NetSalary DECIMAL (12,2) NOT NULL,
Deductions DECIMAL (12,2) NOT NULL DEFAULT 0.00,
Status ENUM('Pending','Processed') NOT NULL,
ProcessedDate DATETIME,
FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID) );
```


Survey/Feedback Table Structure: -

Column Name	Data Type	Constraints
SurveyID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
Title	VARCHAR (200)	NOT NULL
CreatedDate	DATETIME	NOT NULL
CreatedBy	INT	FOREIGN KEY REFERENCES Employee (EmployeeID) NOT NULL
TargetAudience	ENUM ('Department',' Role', 'All')	NOT NULL

```
CREATE TABLE Survey ( SurveyID INT PRIMARY KEY AUTO_INCREMENT,  
Title VARCHAR (200) NOT NULL,  
CreatedDate DATETIME NOT NULL,  
CreatedBy INT NOT NULL,  
TargetAudience ENUM ('Department',' Role',' All') NOT NULL,  
FOREIGN KEY (CreatedBy) REFERENCES Employee (EmployeeID) );
```

Survey Response Table Structure: -

Column Name	Data Type	Constraints
ResponseID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
SurveyID	INT	FOREIGN KEY REFERENCES Survey (SurveyID) NOT NULL
EmployeeID	INT	FOREIGN KEY REFERENCES Employee (EmployeeID) NOT NULL
ResponseData	JSON (if supported) / TEXT	NOT NULL
SubmittedDate	DATETIME	NOT NULL

```
CREATE TABLE SurveyResponse ( ResponseID INT PRIMARY KEY AUTO_INCREMENT,  
SurveyID INT NOT NULL,  
EmployeeID INT NOT NULL,  
ResponseData JSON NOT NULL,  
SubmittedDate DATETIME NOT NULL,  
FOREIGN KEY (SurveyID) REFERENCES Survey(SurveyID),  
FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID) );
```

Performance Review Table Structure: -

Column Name	Data Type	Constraints
-------------	-----------	-------------

ReviewID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
EmployeeID	INT	FOREIGN KEY REFERENCES Employee (EmployeeID) NOT NULL
ReviewCycle	ENUM('Q1','Q2','Q3','Q4','Annual')	NOT NULL
ReviewerID	INT	FOREIGN KEY REFERENCES Employee (EmployeeID) NOT NULL
Rating	DECIMAL (3,2)	NOT NULL CHECK (Rating >= 0.00 AND Rating <= 5.00)
Feedback	TEXT	NULL
Status	ENUM ('Draft', 'Submitted', 'Approved')	NOT NULL

```
CREATE TABLE PerformanceReview ( ReviewID INT PRIMARY KEY AUTO_INCREMENT,
EmployeeID INT NOT NULL,
ReviewCycle ENUM('Q1','Q2','Q3','Q4','Annual') NOT NULL,
ReviewerID INT NOT NULL,
Rating DECIMAL(3,2) NOT NULL CHECK (Rating >= 0.00 AND Rating <= 5.00),
Feedback TEXT,
Status ENUM('Draft','Submitted','Approved') NOT NULL,
FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID),
FOREIGN KEY (ReviewerID) REFERENCES Employee(EmployeeID) );
```

1.02 Employee Dashboard

Employee Table Structure: -

Column	Type	Example Value
EmployeeID	INT PRIMARY KEY	1001
FullName	VARCHAR (100)	'Amit Kumar'
DateOfBirth	DATE	'1990-08-15'
Address	VARCHAR (255)	'123 Green St, Delhi'
ContactNumber	VARCHAR (15)	'+91-9876543210'
Email	VARCHAR (100)	' amit.kumar@email.com '
JobTitle	VARCHAR (50)	'Software Engineer'
Department	VARCHAR (50)	'Engineering'
JoiningDate	DATE	'2021-04-01'
EmploymentType	VARCHAR (20)	'Full-Time'
ProfilePhoto	VARCHAR (255)	'photos/1001.jpg'
ManagerID	INT	1000
Status	VARCHAR (20)	'Active'
EmergencyContact	VARCHAR (100)	'Sunita Kumar, 9991112233'
NationalID/TaxID	VARCHAR (30)	'PANAA1234X'
GDPRConsentStatus	VARCHAR (10)	'Consented'

```
CREATE TABLE Employee ( EmployeeID INT PRIMARY KEY,
FullName VARCHAR (100) NOT NULL,
DateOfBirth DATE, Address VARCHAR (255),
ContactNumber VARCHAR (15),
Email VARCHAR (100) UNIQUE,
JobTitle VARCHAR (50),
Department VARCHAR (50),
JoiningDate DATE,
EmploymentType VARCHAR (20),
ProfilePhoto VARCHAR (255),
ManagerID INT, Status VARCHAR (20),
EmergencyContact VARCHAR (100),
NationalID_TaxID VARCHAR (30) UNIQUE,
GDPRConsentStatus VARCHAR (15) );
```

Attendance Table Structure: -

Column	Type	Example Value
AttendanceID	INT PRIMARY KEY	30245
EmployeeID	INT	1001
Date	DATE	'2025-09-25'
ClockInTime	TIME	'09:01:27'
ClockOutTime	TIME	'17:21:03'
Location	VARCHAR (100)	'28.7041,77.1025'
ShiftType	VARCHAR (20)	'General'
TimesheetDetails	VARCHAR (255)	'8 hours completed'
CorrectionRequestID	INT	NULL
Status	VARCHAR (20)	'Normal'
LeaveIntegrationReference	INT	NULL

```
CREATE TABLE Attendance ( AttendanceID INT PRIMARY KEY, EmployeeID INT NOT NULL,

Date DATE NOT NULL,
ClockInTime TIME,
ClockOutTime TIME,
Location VARCHAR (50),
ShiftType VARCHAR (20),
TimesheetDetails VARCHAR (255),
CorrectionRequestID INT,
Status VARCHAR (20),
LeaveIntegrationReference INT,
```

FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID)

Payslip Table Structure: -

Column	Type	Example Value
PayslipID	INT PRIMARY KEY	501
EmployeeID	INT	1001
Month	INT	9
Year	INT	2025
GrossSalary	DECIMAL (10,2)	85000.00
Deductions	DECIMAL (10,2)	5000.00
NetSalary	DECIMAL (10,2)	80000.00
TaxRegimeSelected	VARCHAR (10)	'New'
DownloadURL	VARCHAR (255)	'payslips/501.pdf'
Form16URL	VARCHAR (255)	'forms/16_1001.pdf'
TaxDeductionDetails	VARCHAR (255)	'PF:1000, PT:2000'

```
CREATE TABLE Payslip ( PayslipID INT PRIMARY KEY,  
EmployeeID INT NOT NULL,  
Month INT NOT NULL,  
Year INT NOT NULL,  
GrossSalary DECIMAL (10, 2),  
Deductions DECIMAL (10, 2),  
NetSalary DECIMAL (10, 2),  
TaxRegimeSelected VARCHAR (10),  
DownloadURL VARCHAR (255),  
Form16URL VARCHAR (255),  
TaxDeductionDetails VARCHAR (255),  
FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID) );
```

Leave Table Structure: -

Column	Type	Example Value
LeaveID	INT PRIMARY KEY	2001
EmployeeID	INT	1001
LeaveType	VARCHAR (30)	'Annual'
StartDate	DATE	'2025-10-05'

EndDate	DATE	'2025-10-09'
Status	VARCHAR (20)	'Pending'
LeaveBalance	INT	12
LeaveHistory	VARCHAR (255)	"
Comments	VARCHAR (255)	'Family event'
AttachmentURL	VARCHAR (255)	"
ApproverID	INT	1000

```
CREATE TABLE Leave ( LeaveID INT PRIMARY KEY, EmployeeID INT NOT NULL,
LeaveType VARCHAR(30), StartDate DATE, EndDate DATE, Status VARCHAR(20), LeaveBalance INT,
LeaveHistory TEXT,
Comments VARCHAR(255),
AttachmentURL VARCHAR(255),
ApproverID INT,
FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID) );
```

Benefits Table Structure: -

Column	Type	Example Value
BenefitID	INT PRIMARY KEY	42
EmployeeID	INT	1001
BenefitType	VARCHAR (30)	'Insurance'
EnrollmentStatus	VARCHAR (20)	'Enrolled'
ClaimID	INT	505
ClaimStatus	VARCHAR (20)	'Approved'
ReceiptAttachmentURL	VARCHAR (255)	'claims/505.pdf'
HealthCardURL	VARCHAR (255)	'cards/1001.pdf'

```
CREATE TABLE Benefits ( BenefitID INT PRIMARY KEY, EmployeeID INT NOT NULL,
BenefitType VARCHAR (30),
EnrollmentStatus VARCHAR (20),
ClaimID INT,
ClaimStatus VARCHAR (20),
ReceiptAttachmentURL VARCHAR (255),
HealthCardURL VARCHAR (255),
FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID) );
```

Performance Table Structure: -

Column	Type	Example Value
--------	------	---------------

PerformanceID	INT PRIMARY KEY	9001
EmployeeID	INT	1001
Goals	TEXT	'["Finish Project X"]'
SelfReviewText	TEXT	'Completed all tasks'
ManagerFeedback	TEXT	'Excellent work'
ReviewPeriod	VARCHAR(30)	'H1 2025'
Score	INT	95
ScoreHistory	TEXT	''

CREATE TABLE Performance (PerformanceID INT PRIMARY KEY,
EmployeeID INT NOT NULL,
Goals TEXT,
SelfReviewText TEXT,
ManagerFeedback TEXT,
ReviewPeriod VARCHAR (30),
Score INT,
ScoreHistory TEXT,
FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID));

Survey & Engagement Table Structure: -

Column	Type	Example Value
SurveyID	INT PRIMARY KEY	3001
SurveyType	VARCHAR(20)	'Pulse'
QuestionSet	TEXT	'["Q1","Q2"]'
ResponseStatus	VARCHAR(20)	'Completed'
AppreciationID	INT	150
RecognitionType	VARCHAR(20)	'Thank You'
RecognizedBy	INT	1002
AnnouncementID	INT	21
AnnouncementType	VARCHAR(20)	'HR Update'
AnnouncementText	VARCHAR(255)	'Holiday on Oct 2nd'

CREATE TABLE SurveyEngagement (SurveyID INT PRIMARY KEY, SurveyType VARCHAR (20),
QuestionSet TEXT,
ResponseStatus VARCHAR (20),
AppreciationID INT,
RecognitionType VARCHAR (30),
RecognizedBy INT,
AnnouncementID INT,

AnnouncementType VARCHAR (30),
AnnouncementText VARCHAR (255),
FOREIGN KEY (RecognizedBy) REFERENCES Employee (EmployeeID));

Support & Policy Documents Table Structure :

Column	Type	Example Value
TicketID	INT PRIMARY KEY	7001
EmployeeID	INT	1001
QueryType	VARCHAR(30)	'Payroll'
Description	VARCHAR(255)	'Payslip not found'
Status	VARCHAR(20)	'Open'
HRContactID	INT	5
PolicyDocumentID	INT	201
DocumentType	VARCHAR(30)	'Handbook'
DownloadLink	VARCHAR(255)	'policies/201.pdf'

CREATE TABLE SupportPolicyDocuments (TicketID INT PRIMARY KEY, EmployeeID INT NOT NULL,
QueryType VARCHAR(50),
Description VARCHAR(255),
Status VARCHAR(20),
HRContactID INT,
PolicyDocumentID INT,
DocumentType VARCHAR(30),
DownloadLink VARCHAR(255),
FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID));

Security & Compliance Table Structure : -

Column	Type	Example Value
AuditLogID	INT PRIMARY KEY	8001
Timestamp	DATETIME	'2025-09-25 10:00:00'
UserID	INT	1001
ActionType	VARCHAR(50)	'Payslip Download'
TransactionDetails	VARCHAR(255)	'PayslipID: 501'
OAuth2/JWTToken	VARCHAR(255)	'eyJ0eXAiOiJKV1QiLCJ...'
DataEncryptionStatus	VARCHAR(30)	'At Rest'
GDPRComplianceStatus	VARCHAR(15)	'Yes'
LaborLawComplianceStatus	VARCHAR(15)	'Yes'

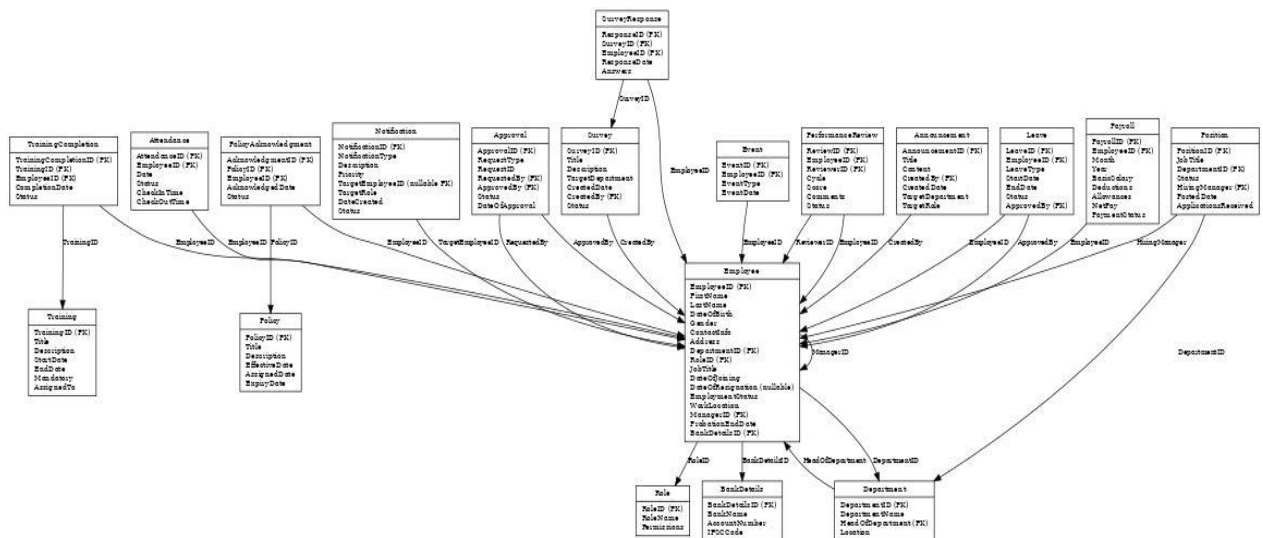
CREATE TABLE SecurityComplianceAudit (AuditLogID INT PRIMARY KEY,
Timestamp DATETIME NOT NULL,
UserID INT NOT NULL,
ActionType VARCHAR (50),
TransactionDetails VARCHAR (255),
OAuth2_JWTToken VARCHAR (512),
DataEncryptionStatus VARCHAR (30),
GDPRComplianceStatus VARCHAR (15),
LaborLawComplianceStatus VARCHAR (15),
FOREIGN KEY (UserID) REFERENCES Employee (EmployeeID));

GDPR and Compliance Table Structure :-

Column	Type	Example Value
ConsentRecord	TEXT	'2024-01-15, v2.1, consented'
DataDeletionRequest	TEXT	'Pending, 2025-09-22 11:55'
AccessToDataRequest	TEXT	'Completed, 2025-08-15 14:30'
DataProcessingActivityLog	TEXT	'Payslip generated 2025-09-24'
LegalHoldStatus	VARCHAR (20)	'None'
RetentionPolicyDetails	TEXT	'3 years after exit'

CREATE TABLE GDPRLaborCompliance (ComplianceID INT PRIMARY KEY AUTO_INCREMENT,
ConsentRecord VARCHAR (255),
DataDeletionRequest VARCHAR (255),
AccessToDataRequest VARCHAR (255),
DataProcessingActivityLog TEXT,
LegalHoldStatus VARCHAR (50),
RetentionPolicyDetails VARCHAR (255));

ER Diagram



1.02 Employee Management

Entities and Attribute: (Hamad Ahamad Sharjil)

Employee Table Structure :-

Field Name	Data Type	Dummy Data (Example)
EmployeeID (PK)	Integer	1, 2
FirstName	VARCHAR	Hamad
LastName	VARCHAR	Ahmad
DOB	DATE	1999-05-12, 1993-08-22
Gender	VARCHAR (20)	Male, Female
Pronouns	VARCHAR	He, She
BloodGroup	VARCHAR	B+, O+
ContactInfo	VARCHAR	0984578945
DateOfJoining	DATE	2025-06-15
Status	VARCHAR	Active, On Notice
Photo	VARCHAR	s3://photos/amit.jpg
DigitalSignature	VARCHAR	s3://photos/amit.jpg

```

CREATE TABLE Employee (
  EmployeeID INT PRIMARY KEY AUTO_INCREMENT,
  FirstName VARCHAR (50),

```

LastName VARCHAR (50),
DOB DATE,
Gender VARCHAR (10),
Pronouns VARCHAR (20),
BloodGroup VARCHAR (5),
ContactInfo VARCHAR (100),
DateOfJoining DATE,
Status VARCHAR (20), -- Active, On Notice, etc.
Photo VARCHAR (255), -- File Path or URL
DigitalSignature VARCHAR (255) -- File Path or URL
);

JobDetails Table Structure :-

Field Name	Data Type	Dummy Data (Example)
JobID (PK)	Integer	101, 102
Department	VARCHAR	IT, HR
Designation	VARCHAR	Software Engineer
Grade	VARCHAR	G5, G7
Location	VARCHAR	Pune, Bangalore
ManagerID (FK)	Integer	1, 2

```
CREATE TABLE JobDetails (  
  JobID INT PRIMARY KEY AUTO_INCREMENT,  
  Department VARCHAR (50),  
  Designation VARCHAR (50),  
  Grade VARCHAR (10),  
  Location VARCHAR (50),  
  ManagerID INT,  
  FOREIGN KEY (ManagerID) REFERENCES Employee (EmployeeID)  
);
```

Compliance Table Structure :-

Field Name	Data Type	Dummy Data (Example)
ComplianceID (PK)	Integer	201, 202
PAN	VARCHAR	ABCDE1234F
Aadhaar	VARCHAR	123456789012
Passport	Integer	M1234567

BackgroundVerificationStatus	VARCHAR	Verified, Pending
------------------------------	---------	-------------------

```
CREATE TABLE Compliance (
  ComplianceID INT PRIMARY KEY AUTO_INCREMENT,
  PAN VARCHAR (10),
  Aadhaar VARCHAR (12),
  Passport VARCHAR (20),
  BackgroundVerificationStatus VARCHAR (20)
);
```

BankDetails Table Structure :-

Field Name	Data Type	Dummy Data (Example)
BankID (PK)	Integer	301, 302
AccountNumber	VARCHAR	12345678901
IFSC	VARCHAR	HDFC0001234
BankName	Integer	HDFC Bank, SBI
UAN	VARCHAR	UAN12345
PFNumber		PF12345
TDSDeclaration		Yes, No

```
CREATE TABLE BankDetails (
  BankID INT PRIMARY KEY AUTO_INCREMENT,
  AccountNumber VARCHAR (20),
  IFSC VARCHAR (11),
  BankName VARCHAR (50),
  UAN VARCHAR (15),
  PFNumber VARCHAR (15),
  TDSDeclaration VARCHAR (10));
```

Document Table Structure :-

Field Name	Data Type	Dummy Data (Example)
DocumentID (PK)	Integer	401, 402
DocumentType	VARCHAR	D, Certification

FilePath	VARCHAR	s3://docs/amit_id.pdf
UploadDate	VARCHAR	2020-06-20, 2021-09-15
ExpiryDate	Date	NULL, 2025-09-15
UploadedBy (FK)	Integer	1, 2

```
CREATE TABLE Document (
  DocumentID INT PRIMARY KEY AUTO_INCREMENT,
  DocumentType VARCHAR (50), -- ID, Joining, Certification, Appraisal, etc.
  FilePath VARCHAR (255),
  UploadDate DATE,
  ExpiryDate DATE,
  UploadedBy INT,
  FOREIGN KEY (UploadedBy) REFERENCES Employee (EmployeeID)
);
```

Role Table Structure :-

Field Name	Data Type	Dummy Data (Example)
RoleID (PK)	Integer	1, 2, 3, 4
RoleName	VARCHAR	Admin, HR, Manager
AccessLevel	VARCHAR	High, Medium, Low

```
CREATE TABLE Role (
  RoleID INT PRIMARY KEY AUTO_INCREMENT,
  RoleName VARCHAR (50),
  AccessLevel VARCHAR (20)
);
```

EmployeeRole (Mapping Table for RBAC) Table Structure :-

Field Name	Data Type	Dummy Data (Example)
EmployeeRoleID	Integer	501, 502
EmployeeID (FK)	Integer	1, 2
RoleID (FK)	Integer	1, 2

```
CREATE TABLE EmployeeRole (

EmployeeRoleId INT PRIMARY KEY AUTO_INCREMENT,
EmployeeID INT,
RoleId INT,
FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID),
FOREIGN KEY (RoleId) REFERENCES Role (RoleId)

);
```

ExitDetails Table Structure : -

Field Name	Data Type	Dummy Data (Example)
ExitID (PK)	nteger	601
EmployeeID (FK)	Integer	2
ResignationDate	Date	2023-08-15
ApprovalStatus	VARCHAR	Approved, Pending
ClearanceChecklist	VARCHAR	Assets:Returned,
RelievingLetter	VARCHAR	s3://exit/priya_relieving.pdf
ExperienceCertificate	VARCHAR	s3://exit/priya_experience.pdf
FnFStatement	VARCHAR	s3://exit/priya_fnf.pdf

```
CREATE TABLE ExitDetails (
ExitID INT PRIMARY KEY AUTO_INCREMENT,
EmployeeID INT,
ResignationDate DATE,
ApprovalStatus VARCHAR (20),
ClearanceChecklist VARCHAR (255), -- JSON or text (Assets, Handover, Feedback, Clearance)
RelievingLetter VARCHAR (255), -- Auto-generated file path
ExperienceCertificate VARCHAR (255),
FnFStatement VARCHAR (255),
FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID)
);
```

OrganizationHierarchy Table Structure : -

Field Name	Data Type	Dummy Data (Example)
------------	-----------	----------------------

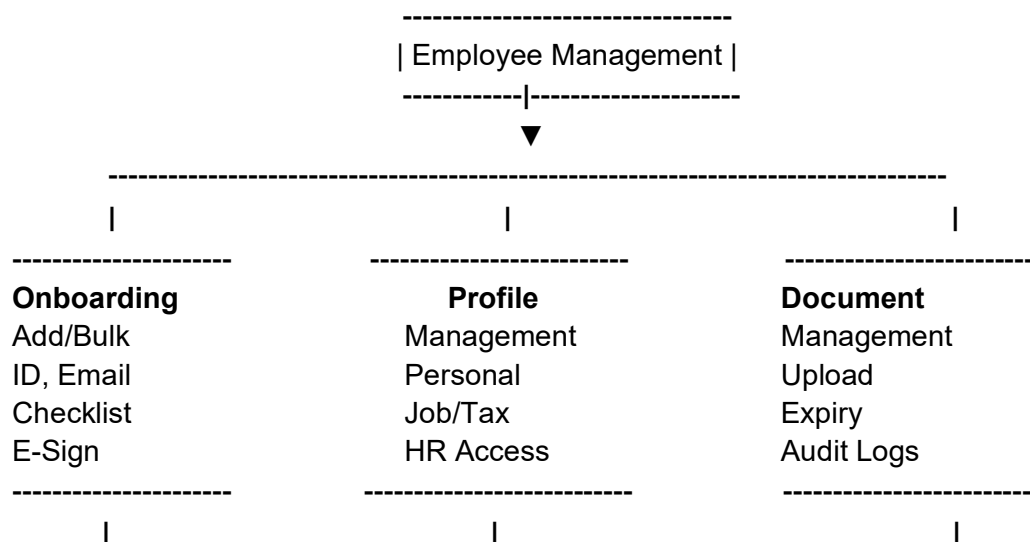
HierarchyID (PK)	Integer	701, 702
EmployeeID (FK)	Integer	1, 2
ReportingManagerID	Integer	2, 1
AlternateManagerID	Integer	NULL
HRBP (FK)	Integer	2, 1

```

CREATE TABLE OrganizationHierarchy (
  HierarchyID INT PRIMARY KEY AUTO_INCREMENT,
  EmployeeID INT,
  ReportingManagerID INT,
  AlternateManagerID INT,
  HRBP INT,
  FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID),
  FOREIGN KEY (ReportingManagerID) REFERENCES Employee (EmployeeID),
  FOREIGN KEY (AlternateManagerID) REFERENCES Employee (EmployeeID),
  FOREIGN KEY (HRBP) REFERENCES Employee (EmployeeID)
);

```

Flow Diagram: -



**Roles &
Access Ctrl**
RBAC
Org Chart

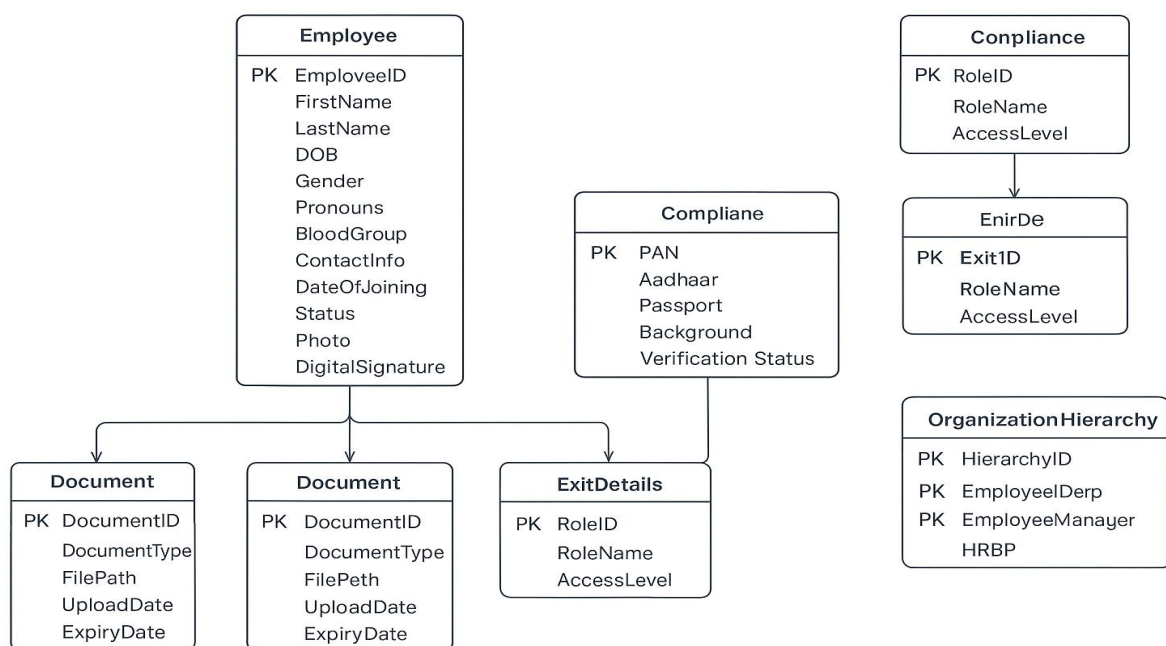
**Job/Dept
Mapping**
Transfers ?
Deputation

Exit Mgmt
-Resignation
Checklist
Auto Letters

Status & Lifecycle Tracking

Active/Resigned/Notice
Auto Alerts
Dashboard Stats

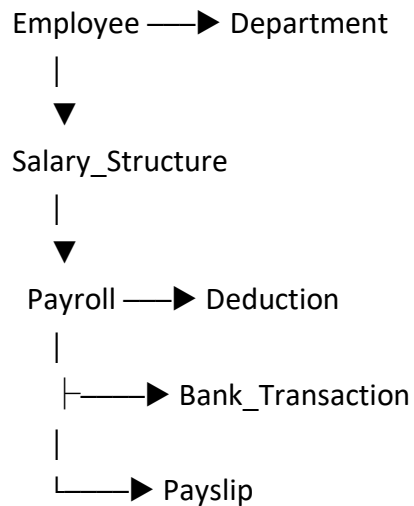
ER Diagram



Module 1.3 Payroll Management

Key Entities and Attribute: (DhirajKumar Saitwal)

Path / Flow Diagram :-



1. Employee Table Structure :-

Column	SQL Type	Example
Employee_ID	INT PRIMARY KEY	101
First_Name	VARCHAR (50)	'Rajesh'
Last_Name	VARCHAR (50)	'Sharma'
Email	VARCHAR (100)	' rajesh.sharma@mail.com '
Phone	VARCHAR (15)	'+91-9876543210'
Department_ID	INT	10

```

CREATE TABLE employee (
  employee_id INT AUTO_INCREMENT PRIMARY KEY,
  first_name VARCHAR(100) NOT NULL,
  last_name VARCHAR(100),
  email VARCHAR(255) UNIQUE NOT NULL,
  phone VARCHAR(20),
  department_id INT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_emp_dept FOREIGN KEY (department_id) REFERENCES
  department(department_id)
  ON DELETE RESTRICT
);

```


2. Department Table Structure :-

Column	SQL Type	Example
Department_ID	INT PRIMARY KEY	10
Department_Name	VARCHAR (100)	'Finance'

```
CREATE TABLE department (  
    department_id INT AUTO_INCREMENT PRIMARY KEY,  
    department_name VARCHAR (100) NOT NULL  
);
```

3. Salary_Structure Table Structure: -

Column	SQL Type	Example
Structure_ID	INT PRIMARY KEY	5001
Basic_Pay	DECIMAL (10,2)	45000.00
HRA	DECIMAL (10,2)	15000.00
Variable_Pay	DECIMAL (10,2)	5000.00
Effective_From	DATE	'2025-01-01'
Effective_To	DATE	'2025-12-31'
Employee_ID	INT	101

```
CREATE TABLE salary_structure (  
    structure_id INT AUTO_INCREMENT PRIMARY KEY,  
    employee_id INT NOT NULL,  
    basic_pay DECIMAL(12,2) NOT NULL DEFAULT 0,  
    hra DECIMAL(12,2) NOT NULL DEFAULT 0,  
    variable_pay DECIMAL(12,2) NOT NULL DEFAULT 0,  
    effective_from DATE NOT NULL,  
    effective_to DATE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
    CONSTRAINT fk_structure_employee FOREIGN KEY (employee_id) REFERENCES employee(employee_id)  
        ON DELETE CASCADE,  
    UNIQUE(employee_id, effective_from)  
);
```

4. Payroll Table Structure:-

Column	SQL Type	Example
Payroll_ID	INT PRIMARY KEY	9001
Month	INT	9
Year	INT	2025
Gross_Salary	DECIMAL (10,2)	65000.00
Net_Salary	DECIMAL (10,2)	60000.00
Deductions_Total	DECIMAL (10,2)	5000.00
Employee_ID	INT	101
Structure_ID	INT	5001

```
CREATE TABLE payroll (
  payroll_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  employee_id INT NOT NULL,
  structure_id INT NOT NULL,
  month TINYINT NOT NULL CHECK (month BETWEEN 1 AND 12),
  year INT NOT NULL,
  gross_salary DECIMAL(14,2) NOT NULL,
  deductions_total DECIMAL(14,2) NOT NULL DEFAULT 0,
  net_salary DECIMAL(14,2) NOT NULL,
  processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  CONSTRAINT fk_payroll_employee FOREIGN KEY (employee_id) REFERENCES employee(employee_id)
    ON DELETE CASCADE,
  CONSTRAINT fk_payroll_structure FOREIGN KEY (structure_id) REFERENCES salary_structure(structure_id)
    ON DELETE RESTRICT,
  UNIQUE(employee_id, month, year)
);
```

5. Deduction Table Structure :-

Column	SQL Type	Example
Deduction_ID	INT PRIMARY KEY	301
Type	VARCHAR (20)	'PF'
Amount	DECIMAL (10,2)	2000.00
Payroll_ID	INT	9001

```
CREATE TABLE deduction (
  deduction_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  payroll_id BIGINT NOT NULL,
  deduction_type VARCHAR (20) NOT NULL,
  amount DECIMAL (12,2) NOT NULL,
  CONSTRAINT fk_deduction_payroll FOREIGN KEY (payroll_id) REFERENCES payroll(payroll_id)
```

ON DELETE CASCADE
);

6. Bank_Transaction: -

Column	SQL Type	Example
Transaction_ID	INT PRIMARY KEY	4501
Payroll_ID	INT	9001
Bank_Name	VARCHAR (50)	'Bank of India'
Account_Number	VARCHAR (20)	'1234567890'
IFSC_Code	VARCHAR (15)	'BKID0000123'
Status	VARCHAR (10)	'Success'
Transaction_Date	DATE	'2025-09-25'

```
CREATE TABLE bank_transaction (  
    transaction_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    payroll_id BIGINT NOT NULL UNIQUE,  
    bank_name VARCHAR(150),  
    account_number VARCHAR(64),  
    ifsc_code VARCHAR(32),  
    status VARCHAR(20) NOT NULL DEFAULT 'INITIATED',  
    transaction_date TIMESTAMP NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
    CONSTRAINT fk_transaction_payroll FOREIGN KEY (payroll_id) REFERENCES payroll(payroll_id)  
    ON DELETE CASCADE  
);
```

7. Payslip Table Structure: -

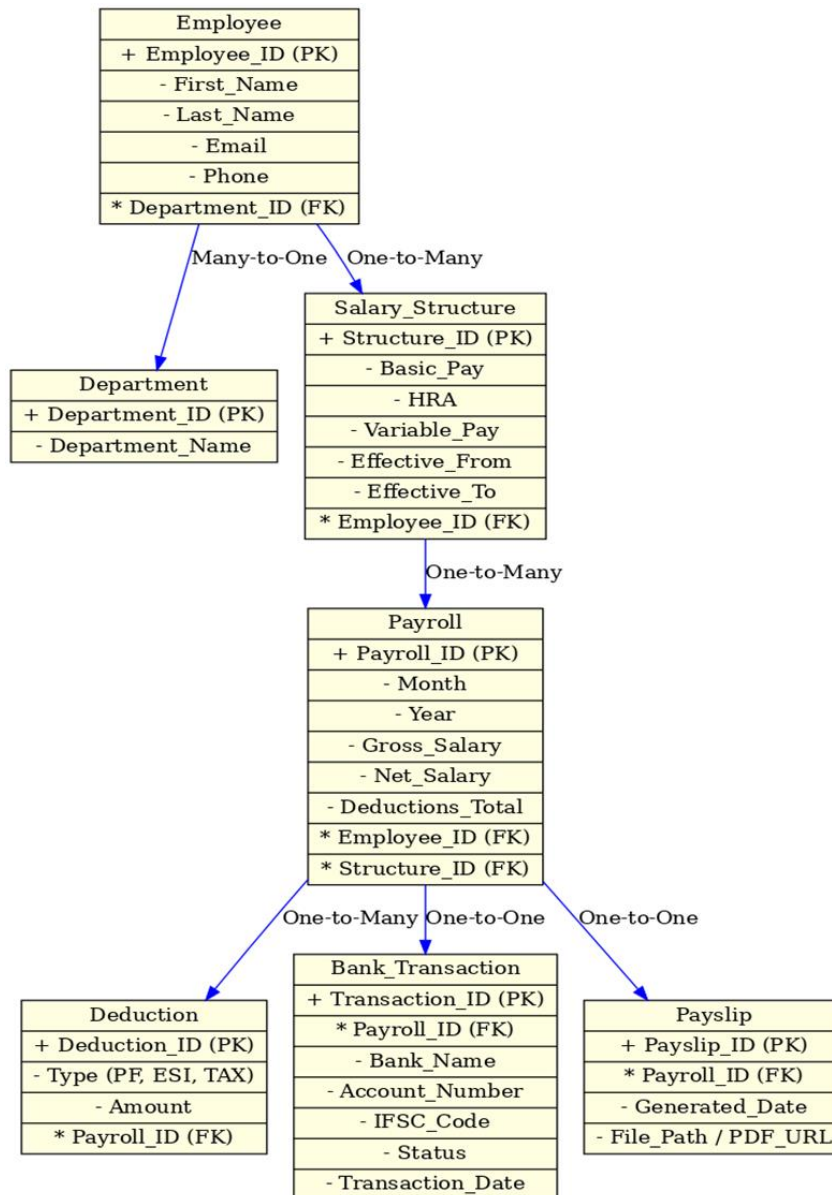
Column	SQL Type	Example
Payslip_ID	INT PRIMARY KEY	7001
Payroll_ID	INT	9001
Generated_Date	DATE	'2025-09-25'
File_Path	VARCHAR (255)	'payslips/7001_sep2025.pdf'

```
CREATE TABLE payslip (  
    payslip_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    payroll_id BIGINT NOT NULL UNIQUE,  
    generated_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    file_url TEXT,
```

CONSTRAINT fk_payslip_payroll FOREIGN KEY (payroll_id) REFERENCES payroll(payroll_id)
ON DELETE CASCADE
)

. Payroll Management ER Diagram and Relationship's

This ERD clearly represents all entities, primary keys (PK), foreign keys (FK), and relationships for the Payroll Management System.



Relationships: -

- **Employee – Department:** Many employees belong to one department.
- **Employee – Salary_Structure:** One employee can have multiple structures over time.
- **Salary_Structure – Payroll:** Payroll is generated based on salary structure.
- **Payroll – Deduction:** One payroll can have multiple deductions.
- **Payroll – Bank_Transaction:** One payroll record maps to one transaction.
- **Payroll – Payslip:** One payroll generates one payslip.

5. API's Structure: -

1. Employee Management API's

Endpoint	Method	Description
/api/employees	POST	Add a new employee
/api/employees	GET	Get all employees
/api/employees/{id}	GET	Get employee by ID

Request (POST): -

```
{
  "first_name": "Amit",
  "last_name": "Sharma",
  "email": "amit.sharma@example.com",
  "phone": "9876543210",
  "department_id": 2
}
```

Response: -

```
{
  "employee_id": 101,
  "message": "Employee created successfully"
}
```

2. Salary Structure API's

Endpoint	Method	Description
/api/salary-structures	POST	Assign salary structure to employee
/api/salary-structures/{id}	GET	Get salary structure by ID

Request (POST): -

```
{ "employee_id": 101, "basic_pay": 30000, "hra": 10000, "variable_pay": 5000,
"effective_from": "2025-10-01"}
```

Response: -

```
{ "structure_id": 201, "message": "Salary structure assigned"}
```

3. Payroll Processing API's

Endpoint	Method	Description
/api/payroll/process	POST	Process monthly payroll
/api/payroll/{employee_id}	GET	Get payroll details for employee

Request (POST): -

```
{ "month": 9, "year": 2025, "employee_ids": [101, 102]}
```

Response: -

```
{ "processed": 2, "payroll_ids": [301, 302], "status": "success"}
```

4. Deductions API's

Endpoint	Method	Description
/api/payroll/{payroll_id}/deductions	GET	Get all deductions for a payroll

Response: -

```
[ { "deduction_type": "PF", "amount": 2000 }, { "deduction_type": "TAX", "amount": 3000 }]
```

5. Bank Transactions API's

Endpoint	Method	Description
/api/bank/transactions	POST	Initiate bank transaction for payroll
/api/bank/transactions/{id}	GET	Get transaction details

Request (POST): -

```
{ "payroll_id": 301, "bank_name": "HDFC Bank", "account_number": "1234567890",  
  "ifsc_code": "HDFC0001234"}
```

Response: -

```
{ "transaction_id": 501, "status": "SUCCESS", "transaction_date": "2025-09-24T10:15:00Z"}
```

6. Payslip API's

Endpoint	Method	Description
/api/payslips/generate	POST	Generate payslip for payroll
/api/payslips/{id}	GET	Get payslip details

Request (POST): -

```
{ "payroll_id": 301}
```

Response: -

```
{ "payslip_id": 601, "file_url": "https://s3.aws.com/payslips/301-sep-2025.pdf",  
  "generated_date": "2025-09-24T10:30:00Z"}
```

Module 1.4 Time and Attendance Management

Key Entities and Attribute: (Suraj Sakarde)

1. Attendance :-

Column Name	Data Type	Constraints
id(pk)	Integer	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
employeeId(FK)	String	FOREIGN KEY REFERENCES Employee (EmployeeId)
Date	Date	NOT NULL
checkInTime	DateTime	NOT NULL
checkOutTime	DateTime	NOT NULL
workLocation	String	NOT NULL
bioMetricId	Integer	NOT NULL
geolocation(geotag)	String	NOT NULL
IsManual	Boolean	NOT NULL
Shift (Gen, Day,Night)	String	NOT NULL
lateCheckInTime	DateTime	NOT NULL
earlyCheckOutTime	DateTime	NOT NULL
Attendance Scheme	String	NOT NULL
Work Hours	String	NOT NULL
Work HoursIn Shift Time	String	NOT NULL
Status	String	NOT NULL
Shift Timings (Session1)	String	NOT NULL
Shift Timings (Session2)	String	NOT NULL

```
CREATE TABLE Attendance (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  employeeId VARCHAR (50) NOT NULL,  
  Date DATE NOT NULL,  
  checkInTime DATETIME NOT NULL,  
  checkOutTime DATETIME NOT NULL,  
  workLocation VARCHAR (100) NOT NULL,  
  bioMetricId INT NOT NULL,  
  geolocation VARCHAR (255) NOT NULL,  
  IsManual BOOLEAN NOT NULL,  
  Shift VARCHAR (20) NOT NULL, -- (Gen, Day, Night)  
  lateCheckInTime DATETIME NOT NULL,  
  earlyCheckOutTime DATETIME NOT NULL,  
  AttendanceScheme VARCHAR (100) NOT NULL,  
  WorkHours VARCHAR (50) NOT NULL,  
  WorkHoursInShiftTime VARCHAR (50) NOT NULL,  
  Status VARCHAR (50) NOT NULL,  
  ShiftTimingsSession1 VARCHAR (50) NOT NULL,  
  ShiftTimingsSession2 VARCHAR (50) NOT NULL CONSTRAINT fk_employee FOREIGN KEY (employeeId)  
  REFERENCES Employee(employeeId));
```


2. Shift: -

Column Name	Data Type	Constraints
Id(pk)	Integer	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
shiftName	String	NOT NULL
startTime	DateTime	NOT NULL
endTime	DateTime	NOT NULL
breakDuration	String	NOT NULL
isNightShift	Boolean	NOT NULL

```
CREATE TABLE Shift (  
  Id INT PRIMARY KEY AUTO_INCREMENT,  
  shiftName VARCHAR (50) NOT NULL,  
  startTime DATETIME NOT NULL,  
  endTime DATETIME NOT NULL,  
  breakDuration VARCHAR (20) NOT NULL,  
  isNightShift BOOLEAN NOT NULL  
);
```

3. Roster : -

Column Name	Data Type	Constraints
Id (PK)	Integer	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
employeeId(FK)	String	FOREIGN KEY REFERENCES Employee (EmployeeId)
shiftId(FK)	Integer	FOREIGN KEY REFERENCES shiftId (ShiftId)
date	Date	NOT NULL
Status (Active,terminated,on leave)	String	NOT NULL

```
CREATE TABLE Roster (  
  Id INT PRIMARY KEY AUTO_INCREMENT,  
  employeeId VARCHAR (50) NOT NULL,  
  shiftId INT NOT NULL,  
  date DATE NOT NULL,  
  Status VARCHAR (20) NOT NULL CHECK (Status IN ('Active', 'Terminated', 'On Leave')),  
  CONSTRAINT fk_employee_roster FOREIGN KEY (employeeId) REFERENCES Employee(employeeId),  
  CONSTRAINT fk_shift_roster FOREIGN KEY (shiftId) REFERENCES Shift (Id)  
);
```

4. Holiday: -

Column Name	Data Type	Constraints
Id(pk)	Integer	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
name	String	NOT NULL
date	Date	NOT NULL
Type (National, company specific, optional)	String	NOT NULL

```
CREATE TABLE Holiday (  
  Id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR (100) NOT NULL,  
  date DATE NOT NULL UNIQUE,  
  Type VARCHAR (30) NOT NULL CHECK (Type IN ('National', 'Company Specific', 'Optional'))  
);
```

5. Timesheet : -

Column Name	Data Type	Constraints
Id(pk)	Integer	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
employeeId(fk)	String	FOREIGN KEY REFERENCES Employee (EmployeeId)
projectCode(fk)	String	FOREIGN KEY REFERENCES Project
TaskDescription	String	NULL
hoursWorked	String	NOT NULL
approvalStatus	String	NOT NULL
date	Date	NOT NULL

```
CREATE TABLE Timesheet (  
  Id INT PRIMARY KEY AUTO_INCREMENT,  
  employeeId VARCHAR (50) NOT NULL,  
  projectCode VARCHAR (50) NOT NULL,  
  TaskDescription VARCHAR (255),  
  hoursWorked VARCHAR (20) NOT NULL,  
  approvalStatus VARCHAR (20) NOT NULL CHECK (approvalStatus IN ('Pending', 'Approved', 'Rejected')),  
  date DATE NOT NULL,  
  CONSTRAINT fk_employee_timesheet FOREIGN KEY (employeeId) REFERENCES Employee(employeeId),  
  CONSTRAINT fk_project_timesheet FOREIGN KEY (projectCode) REFERENCES Project(projectCode)  
);
```

6. Project(for timesheet linking) :-

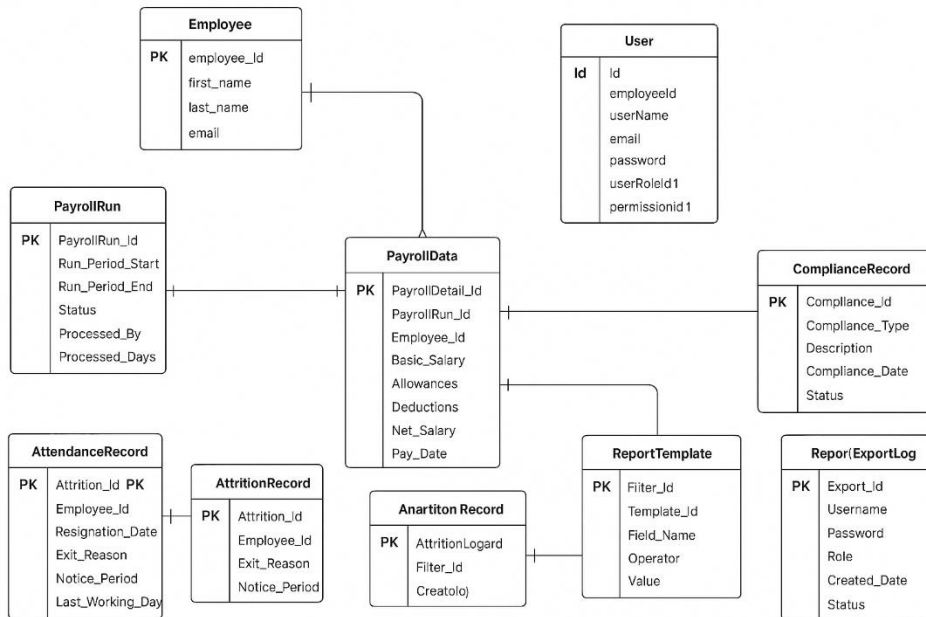
Column Name	Data Type	Constraints
Id(pk)	Integer	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
projectCode	String	NOT NULL
projectName	String	NOT NULL
managerId(fk)	Integer	FOREIGN KEY REFERENCES Employee (EmployeeId)

```
CREATE TABLE Project (  
  Id INT PRIMARY KEY AUTO_INCREMENT,  
  projectCode VARCHAR (50) NOT NULL UNIQUE,  
  projectName VARCHAR (100) NOT NULL,  
  managerId VARCHAR (50) NOT NULL,  
  CONSTRAINT fk_manager FOREIGN KEY (managerId) REFERENCES Employee(employeeId)  
);
```

Entity Relationships

- **Employee ↔ Attendance** → One employee has many attendance records.
- **Employee ↔ Roster ↔ Shift** → Roster assigns employees to shifts.
- **Shift ↔ Roster** → One shift can be assigned to many employees.
- **Holiday** is standalone but referenced in **Attendance/Leave** modules.

Employee ↔ Timesheet ↔ Project → Employee fills timesheet against a project

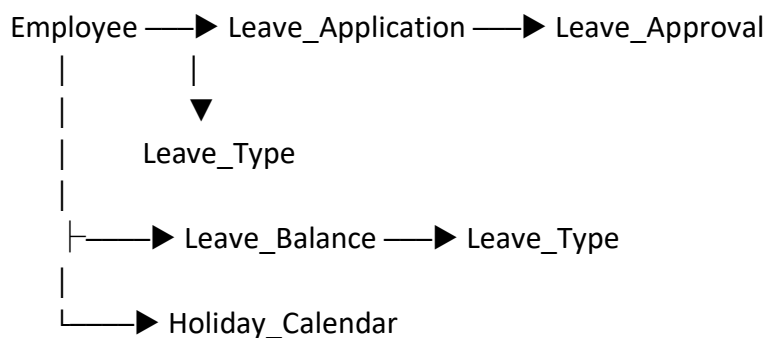


Module 1.5 Leave Management

Key Entities and Attribute: (DhirajKumar Saitwal)

This document describes the Leave Management System as per client requirements. It includes entities, attributes, ER diagram (textual), MySQL table structures, and API structures with request/response examples.

Path / Flow Diagram



1. Employee Table Structure: -

Column Name	Data Type	Constraints
Employee_ID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
First_Name	VARCHAR (50)	NOT NULL
Last_Name	VARCHAR (50)	NOT NULL
Email	VARCHAR (100)	UNIQUE, NOT NULL
Phone	VARCHAR (15)	UNIQUE, NOT NULL
Department_ID	INT	FOREIGN KEY (References Department)

```
CREATE TABLE Employee ( Employee_ID INT PRIMARY KEY AUTO_INCREMENT NOT NULL,  
First_Name VARCHAR (50) NOT NULL,  
Last_Name VARCHAR (50) NOT NULL,  
Email VARCHAR (100) UNIQUE NOT NULL,  
Phone VARCHAR (15) UNIQUE NOT NULL,  
Department_ID INT)
```

2. Leave Type Table Structure: -

Column Name	Data Type	Constraints
Leave_Type_ID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
Type_Name	VARCHAR (50)	NOT NULL
Max_Days_Per_Year	INT	NOT NULL
Carry_Forward	VARCHAR (3)	CHECK (Carry_Forward IN ('Yes', 'No'))
Encashment_Allowed	VARCHAR (3)	CHECK (Encashment_Allowed IN ('Yes', 'No'))

```
CREATE TABLE Leave_Type ( Leave_Type_ID INT PRIMARY KEY AUTO_INCREMENT NOT NULL,  
Type_Name VARCHAR (50) NOT NULL,  
Max_Days_Per_Year INT NOT NULL,  
Carry_Forward VARCHAR (3) CHECK (Carry_Forward IN ('Yes', 'No')),  
Encashment_Allowed VARCHAR (3) CHECK (Encashment_Allowed IN ('Yes', 'No')) );
```

3. Leave_Application Table Structure

Column Name	Data Type	Constraints
Application_ID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
Employee_ID	INT	FOREIGN KEY REFERENCES Employee (Employee_ID), NOT NULL
Leave_Type_ID	INT	FOREIGN KEY REFERENCES Leave_Type(Leave_Type_ID), NOT NULL
Start_Date	DATE	NOT NULL

End_Date	DATE	NOT NULL
Total_Days	INT	NOT NULL
Status	VARCHAR (20)	CHECK (Status IN ('Pending', 'Approved', 'Rejected')) NOT NULL
Comments	VARCHAR (200)	NULL
Attachment_URL	VARCHAR (200)	NULL
Applied_Date	DATE	NOT NULL

```

CREATE TABLE Leave_Application ( Application_ID INT PRIMARY KEY AUTO_INCREMENT NOT NULL,
Employee_ID INT NOT NULL
, Leave_Type_ID INT NOT NULL,
Start_Date DATE NOT NULL,
End_Date DATE NOT NULL,
Total_Days INT NOT NULL,
Status VARCHAR (20) CHECK (Status IN ('Pending', 'Approved', 'Rejected')) NOT NULL,
Comments VARCHAR (200),
Attachment_URL VARCHAR (200),
Applied_Date DATE NOT NULL,
FOREIGN KEY (Employee_ID) REFERENCES Employee (Employee_ID),
FOREIGN KEY (Leave_Type_ID) REFERENCES Leave_Type(Leave_Type_ID) );

```

4. Leave_Approval Table Structure

Column Name	Data Type	Constraints
Approval_ID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
Application_ID	INT	FOREIGN KEY REFERENCES Leave_Application(Application_ID), NOT NULL
Approver_ID	INT	FOREIGN KEY REFERENCES Employee (Employee_ID), NOT NULL
Approval_Level	INT	CHECK (Approval_Level IN (1,2,3)) NOT NULL
Status	VARCHAR (20)	CHECK (Status IN ('Pending', 'Approved', 'Rejected')) NOT NULL
Remarks	VARCHAR (200)	NULL

```

CREATE TABLE Leave_Approval ( Approval_ID INT PRIMARY KEY AUTO_INCREMENT NOT NULL,
Application_ID INT NOT NULL,
Approver_ID INT NOT NULL,
Approval_Level INT CHECK (Approval_Level IN (1,2,3)) NOT NULL,
Status VARCHAR(20) CHECK (Status IN ('Pending', 'Approved', 'Rejected')) NOT NULL,
Remarks VARCHAR(200),
FOREIGN KEY (Application_ID) REFERENCES Leave_Application(Application_ID),
FOREIGN KEY (Approver_ID) REFERENCES Employee (Employee_ID) );

```

4. Leave_Balance Table Structure :-

Column Name	Data Type	Constraints
Balance_ID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT

Employee_ID	INT	FOREIGN KEY REFERENCES Employee (Employee_ID), NOT NULL
Leave_Type_ID	INT	FOREIGN KEY REFERENCES Leave_Type(Leave_Type_ID), NOT NULL
Year	INT	NOT NULL
Total_Allocated	INT	NOT NULL
Total_Used	INT	DEFAULT 0
Carry_Forwarded	INT	DEFAULT 0
Encashed	INT	DEFAULT 0

```
CREATE TABLE Leave_Balance ( Balance_ID INT PRIMARY KEY AUTO_INCREMENT NOT NULL,
Employee_ID INT NOT NULL,
Leave_Type_ID INT NOT NULL,
Year INT NOT NULL,
Total_Allocated INT NOT NULL,
Total_Used INT DEFAULT 0,
Carry_Forwarded INT DEFAULT 0,
Encashed INT DEFAULT 0,
FOREIGN KEY (Employee_ID) REFERENCES Employee(Employee_ID),
FOREIGN KEY (Leave_Type_ID) REFERENCES Leave_Type(Leave_Type_ID) );
```

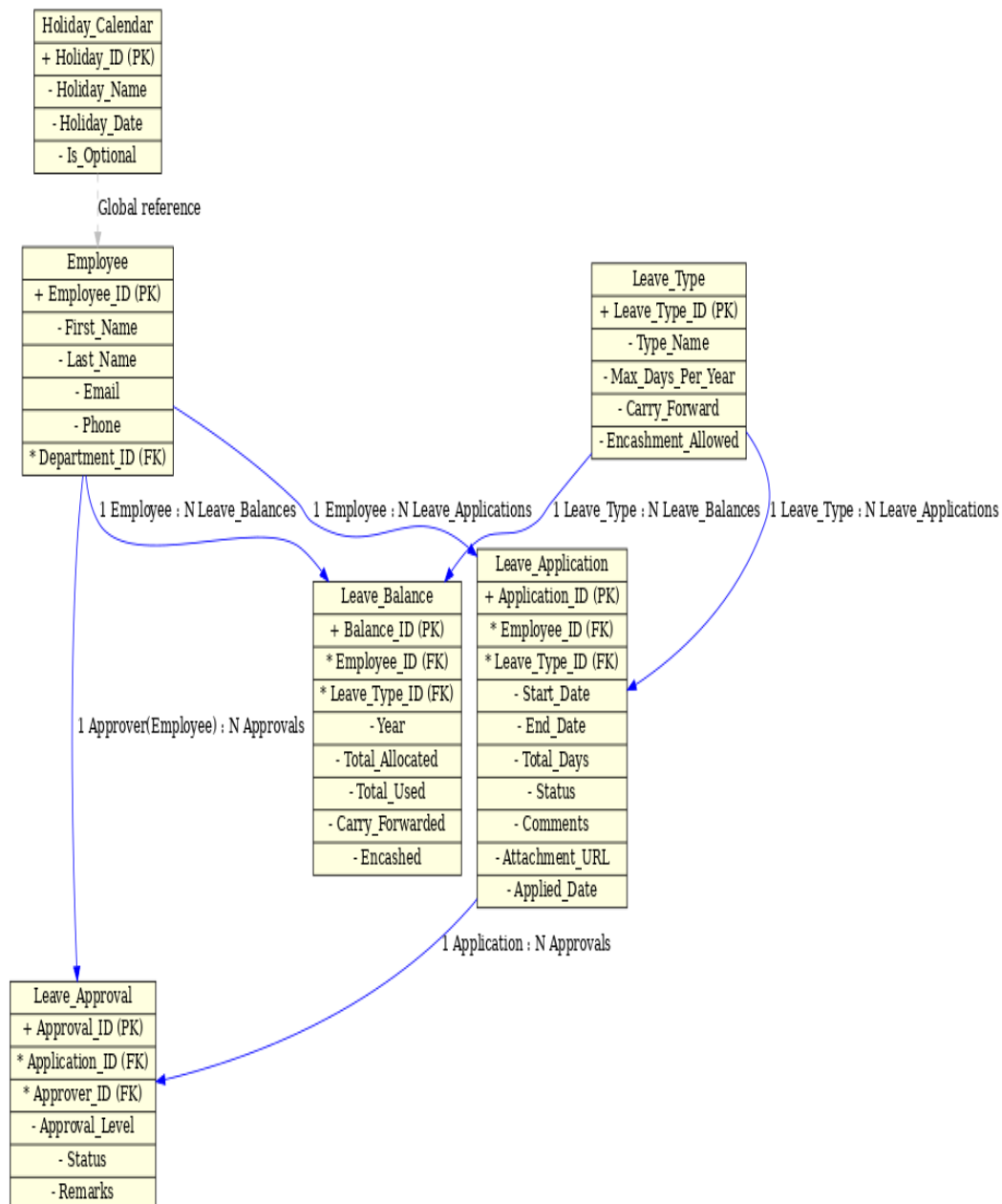
5. Holiday_Calendar Table Structure

Column Name	Data Type	Constraints
Holiday_ID	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
Holiday_Name	VARCHAR(100)	NOT NULL
Holiday_Date	DATE	NOT NULL
Is_Optional	VARCHAR(3)	CHECK (Is_Optional IN ('Yes','No')) NOT NULL

```
CREATE TABLE Holiday_Calendar ( Holiday_ID INT PRIMARY KEY AUTO_INCREMENT NOT NULL,
Holiday_Name VARCHAR(100) NOT NULL,
Holiday_Date DATE NOT NULL,
Is_Optional VARCHAR(3) CHECK (Is_Optional IN ('Yes','No')) NOT NULL );
```

Leave Management ER Diagram and Relationship's

This ERD clearly represents all entities, primary keys (PK), foreign keys (FK), and relationships for the Leave Management System.



5. API's Structure

1. Leave Types API's

Endpoint	Method	Description
/api/leave-types	POST	Add leave type
/api/leave-types	GET	Get all leave types

Response: -


```
[
  { "leave_type_id": 1, "type_name": "Sick", "max_days_per_year": 12 },
  { "leave_type_id": 2, "type_name": "Casual", "max_days_per_year": 10 }
]
```

2. Leave Application API's

Endpoint	Method	Description
/api/leave/apply	POST	Apply for leave
/api/leave/{id}	GET	Get leave application

Request (POST): -

```
{
  "employee_id": 101,
  "leave_type_id": 1,
  "start_date": "2025-10-05",
  "end_date": "2025-10-07",
  "total_days": 3,
  "comments": "Medical leave",
  "attachment_url": "https://s3.aws.com/leaves/medical-certificate.pdf"
}
```

Response: -

```
{
  "application_id": 501,
  "status": "Pending"
}
```

3. Leave Approval API's

Endpoint	Method	Description
/api/leave/approve	POST	Approve/Reject leave

Request (POST): -

```
{
  "application_id": 501,
  "approver_id": 201,
```

```
"approval_level": 1,  
"status": "Approved",  
"remarks": "Manager approved"  
}
```

Response: -

```
{  
  "approval_id": 1001,  
  "status": "Approved"  
}
```

4. Leave Balance API's

Endpoint	Method	Description
/api/leave/balance/{employee_id}	GET	Get employee leave balance

Response: -

```
[  
  { "leave_type": "Sick", "allocated": 12, "used": 3, "carry_forwarded": 2 },  
  { "leave_type": "Casual", "allocated": 10, "used": 1, "carry_forwarded": 0 }  
]
```

5. Holiday Calendar API's

Endpoint	Method	Description
/api/holidays	GET	Get holiday List
/api/holidays	POST	Add holiday

Response: - [

```
{ "holiday_id": 1, "holiday_name": "Diwali", "holiday_date": "2025-11-12" },  
{ "holiday_id": 2, "holiday_name": "Christmas", "holiday_date": "2025-12-25" }  
]
```

Module 1.6 Benefit and Compliance Management

Key Entities and Attribute:(Rohit Mahajan)

1. Employee: -

Column Name	Data Type	Constraints
employee_id	INT (PK)	Primary Key, Auto Increment
first_name	VARCHAR (50)	Not Null
last_name	VARCHAR (50)	Not Null
date_of_joining	DATE	Not Null
department_id	INT (FK)	Foreign Key → Department(department_id)
status	ENUM ('Active', 'Inactive')	Default 'Active'

• 2. Benefits_Eligibility : -

Column Name	Data Type	Constraints
eligibility_id	INT (pk)	Primary Key, Auto Increment
employee_id	INT (fk)	Foreign Key → Employee(employee_id), Not Null
esi_eligible	BOOLEAN	Default FALSE
pf_eligible	BOOLEAN	Default FALSE
insurance_eligible	BOOLEAN	Default FALSE
gratuity_eligible	BOOLEAN	Default FALSE
tds_applicable	BOOLEAN	Default FALSE
created_at	DATETIME	Default Current_Timestamp

• 2. Tax Configuration: -

Column Name	Data Type	Constraints
-------------	-----------	-------------

config_id	INT	Primary Key, Auto Increment
employee_id	INT	Foreign Key → Employee(employee_id), Not Null
tax_regime	VARCHAR(20)	Check (Old / New), Not Null
financial_year	VARCHAR(10)	Not Null
taxable_income	DECIMAL(12,2)	Not Null
tax_amount	DECIMAL(12,2)	Not Null
updated_at	TIMESTAMP	Default Current_Timestamp on Update Current_Timestamp

• 3. Compliance Documents : -

Column Name	Data Type	Constraints
form_id	INT	Primary Key, Auto Increment
employee_id	INT	Foreign Key → Employee(employee_id), Not Null
form_type	VARCHAR(50)	Check (Form 16 / Form 24Q)
generated_date	DATE	Not Null
file_path	VARCHAR(255)	Not Null
status	VARCHAR(20)	Default 'Generated'

• 4. Benefit_Provider : -

Column Name	Data Type	Constraints
document_id	INT	Primary Key, Auto Increment
employee_id	INT	Foreign Key → Employee(employee_id), Not Null
provider_name	VARCHAR(150)	Not Null
document_type	VARCHAR(100)	Not Null
document_path	VARCHAR(255)	Not Null
upload_date	TIMESTAMP	Default Current_Timestamp
status	VARCHAR(20)	Default 'Active'

Relationships:

- **Employee to Benefits_Eligibility:**
 - **Type:** One-to-Many (1:M)
 - **Description:** One **Employee** can be eligible for many **Benefits**.
- **Employee to Tax_Configuration:**
 - **Type:** One-to-One (1:1)
 - **Description:** One **Employee** has exactly one **Tax Configuration**.
- **Employee to Compliance_Document:**
 - **Type:** One-to-Many (1:M)
 - **Description:** One **Employee** can have many **Compliance Documents** generated.
- **Benefits_Eligibility to Benefit_Provider:**
 - **Type:** Many-to-One (M:1)
 - **Description:** Many different types of **Benefits** (e.g., various insurance plans) can be provided by one **Benefit Provider**.

❖ Flow Chart

Start

|

v

Employee Enrollment --> Benefit Eligibility Check --> Document Upload & Verification

|

|

v

v

Track Contributions (ESI/PF/Gratuity/Insurance)

|

v

Auto-generate Compliance Forms (Form16, Form24Q)

|

v

Tax Configuration (New/Old Regime)

|

v

Notifications & Alerts

|

v

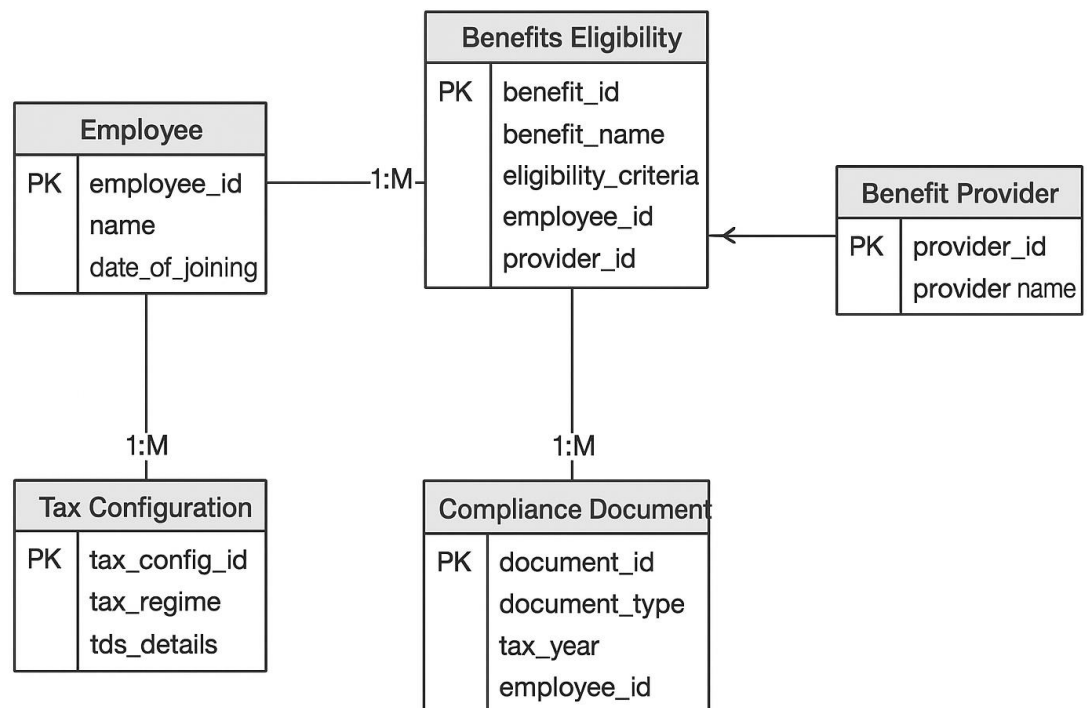
Reports & Analytics

|

v

End

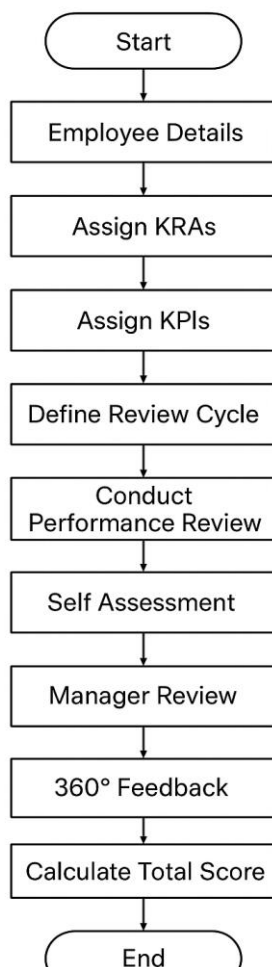
- **ER Diagram**



1.7 Performance Management

Key Entities and Attribute: (Shreya Chaudhri)

Flow Chart



1. Employee :-

Column Name	Data Type	Constraints
Employee_Id (PK)	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
First_Name	VARCAHR (50)	NOT NULL
Last_Name	VARCHAR (50)	NOT NULL
Email	VARCHAR (50)	UNIQUE NOT NULL

Department_Id (FK)	INT	NOT NULL
Role_Id (FK)	INT	NOT NULL

```
CREATE TABLE Employee (
    Employee_Id INT PRIMARY KEY,
    First_Name VARCHAR (50) NOT NULL,
    Last_Name VARCHAR (50) NOT NULL,
    Email VARCHAR (100) UNIQUE NOT NULL,
    Department_Id INT,
    Role_Id INT,
    CONSTRAINT fk_department FOREIGN KEY (Department_Id) REFERENCES Department
    (Department_Id),
    CONSTRAINT fk_role FOREIGN KEY (Role_Id) REFERENCES Role (Role_Id)
);
```

2. Department :-

Column Name	Data Type	Constraints
Department_Id (PK)	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
Department_Name	VARCHAR (100)	NOT NULL UNIQUE

```
CREATE TABLE Department (
    Department_Id INT PRIMARY KEY,
    Department_Name VARCHAR (100) NOT NULL
);
```

3. KRA(Key Result Area) :-

Column Name	Data Type	Constraints
KRA_Id(PK)	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
Title	VARCHAR (100)	NOT NULL
Description	VARCHAR (255)	NOT NULL
Department_Id(FK)	INT	NOT NULL
Role_Id(FK)	INT	NOT NULL

```
CREATE TABLE KRA (
    KRA_Id INT PRIMARY KEY,
    Title VARCHAR (100) NOT NULL,
    Description VARCHAR (255),
    Department_Id INT,
    Role_Id INT,
```



```

CONSTRAINT fk_department_kra FOREIGN KEY (Department_Id) REFERENCES Department
(Department_Id),
CONSTRAINT fk_role_kra FOREIGN KEY (Role_Id) REFERENCES Role (Role_Id)
);

```

4. KPI (Key Performance Indicator) : -

Column Name	Data Type	Constraints
KPI_Id(Pk)	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
Title	VARCHAR (50)	NOT NULL
Description	VARCHAR (255)	NOT NULL
Weightage	DECIMAL (5,2)	CHECK (Weightage >= 0 AND Weightage <= 100)
Target_Value	VARCHAR (50)	NOT NULL
KRA_Id(FK)	INT	NOT NULL

```

CREATE TABLE KPI (
    KPI_Id INT PRIMARY KEY,
    Title VARCHAR (100) NOT NULL,
    Description VARCHAR (255),
    Weightage DECIMAL (5,2),
    Target_Value VARCHAR (50),
    KRA_Id INT,
    CONSTRAINT fk_kra FOREIGN KEY (KRA_Id) REFERENCES KRA(KRA_Id)
);

```

5. Review Cycle : -

Column Name	Data Type	Constraints
Cycle_Id(PK)	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
Review_Id(FK)	INT	
Name	VARCHAR (100)	NOT NULL UNIQUE
StartDate DATETIME	DATETIME	NOT NULL
EndDate DATETIME	DATETIME	NOT NULL

```

CREATE TABLE Review_Cycle (
    Cycle_Id INT PRIMARY KEY,
    Review_Id INT,
    Name VARCHAR (100) NOT NULL,
    StartDate DATETIME NOT NULL,
    EndDate DATETIME NOT NULL,
    CONSTRAINT fk_review FOREIGN KEY (Review_Id) REFERENCES Review (Review_Id)
);

```

);

6. Performance Review : -

Column Name	Data Type	Constraints
Review_Id(PK)	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
Employee_Id(FK)	INT	NOT NULL
Cycle_Id(FK)	INT	NOT NULL
Total_Score	DECIMAL (5,2)	CHECK (Total_Score >= 0 AND Total_Score <= 100)

```
CREATE TABLE Performance_Review (  
    Review_Id INT PRIMARY KEY,  
    Employee_Id INT,  
    Cycle_Id INT,  
    Total_Score DECIMAL (5,2),  
    CONSTRAINT fk_employee_review FOREIGN KEY (Employee_Id) REFERENCES Employee (Employee_Id),  
    CONSTRAINT fk_cycle_review FOREIGN KEY (Cycle_Id) REFERENCES Review_Cycle(Cycle_Id)  
);
```

7. Self Assessment : -

Column Name	Data Type	Constraints
SelfAssessment_Id(PK)	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
Review_Id(FK)	INT	NOT NULL
Comments	VARCHAR (255)	
Rating	DECIMAL (3,2)	CHECK (Rating >= 0 AND Rating <= 5)

```
CREATE TABLE Self_Assessment (  
    SelfAssessment_Id INT PRIMARY KEY,  
    Review_Id INT,  
    Comments VARCHAR (255),  
    Rating DECIMAL (3,2),  
    CONSTRAINT fk_review_self FOREIGN KEY (Review_Id) REFERENCES Performance_Review(Review_Id)  
);
```

8. Manager Review : -

Column Name	Data Type	Constraints
-------------	-----------	-------------

ManagerReview_Id(PK)	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
Review_Id(FK)	INT	NOT NULL
Comments	VARCHAR (255)	NOT NULL
Rating	DECIMAL (3,2)	CHECK (Rating >= 0 AND Rating <= 5)

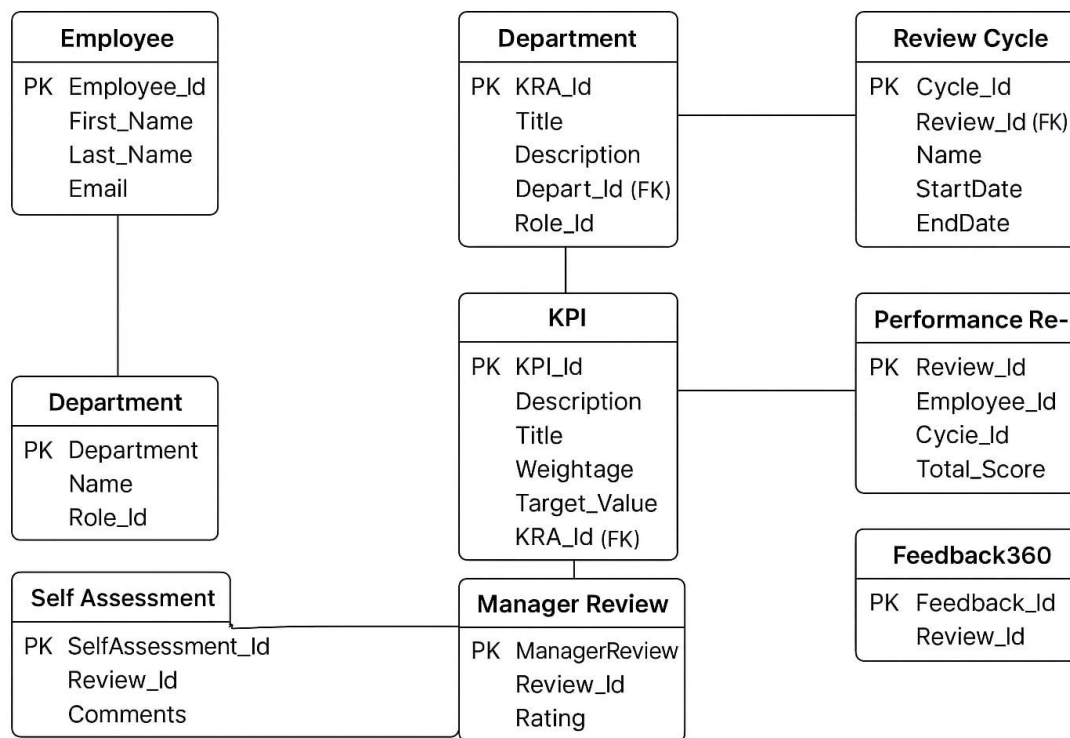
```
CREATE TABLE Manager_Review (
    ManagerReview_Id INT PRIMARY KEY,
    Review_Id INT,
    Comments VARCHAR (255),
    Rating DECIMAL (3,2),
    CONSTRAINT fk_review_manager FOREIGN KEY (Review_Id) REFERENCES Performance_Review(Review_Id)
);
```

9. Feedback360: -

Column Name	Data Type	Constraints
Feedback_Id(PK)	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
Feedback_Id(FK)	INT	NOT NULL

```
CREATE TABLE Feedback360 (
    Feedback_Id INT PRIMARY KEY,
    Review_Id INT,
    CONSTRAINT fk_review_feedback FOREIGN KEY (Review_Id) REFERENCES Performance_Review(Review_Id)
);
```

ER Diagram



Relationships: -

- **Employee ↔ Department:** Many Employees belong to one Department.
- **Employee ↔ Role:** Many Employees can have one Role.
- **KRA ↔ Department:** Many KRAs belong to one Department.
- **KRA ↔ Role:** Many KRAs are associated with one Role.
- **KPI ↔ KRA:** Many KPIs belong to one KRA.
- **Performance Review ↔ Employee:** Many Performance Reviews belong to one Employee.
- **Performance Review ↔ Review Cycle:** Many Reviews belong to one Review Cycle.
- **Review Cycle ↔ Performance Review:** One Review Cycle contains many Reviews.
- **Self-Assessment ↔ Performance Review:** One Self-Assessment per Performance Review.
- **Manager Review ↔ Performance Review:** One Manager Review per Performance Review.
- **Feedback360 ↔ Performance Review:** One Feedback360 per Performance Review (assumed, can be many-to-one if multiple feedback entries are allowed).

Module 1.8 Report and Analysis Management

Key Entities and Attribute: (Harshada Bundelkar)

1. PayrollRun: -

Column Name	Data Type	Constraints
PayrollID (PK)	Integer	PRIMARY KEY
CompanyID (FK)	Integer	NOT NULL
PeriodStart	DATE	NOT NULL
PeriodEnd	DATE	NOT NULL
GeneratedOn	DATE	NOT NULL
Status	VARCHAR/String	NOT NULL

```
CREATE TABLE PayrollRun (
    PayrollID INT PRIMARY KEY,
    CompanyID INT NOT NULL,
    PeriodStart DATE NOT NULL,
    PeriodEnd DATE NOT NULL,
    GeneratedOn DATE NOT NULL,
    Status VARCHAR(20) NOT NULL CHECK (Status IN ('Draft','Processed','Paid')),
    CONSTRAINT chk_period CHECK (PeriodEnd >= PeriodStart),
    FOREIGN KEY (CompanyID) REFERENCES Company(CompanyID)
);
```

2. PayrollDetail: -

Column Name	Data Type	Constraints
PayrollDetailID (PK)	Integer	PRIMARY KEY
PayrollID (FK)	Integer	NOT NULL
EmpID (FK)	Integer	NOT NULL
ComponentName	VARCHAR/String	NOT NULL
Amount	DECIMAL(10,2)	NOTNULL

```
CREATE TABLE PayrollDetail (
    PayrollDetailID INT PRIMARY KEY,
    PayrollID INT NOT NULL,
    EmpID INT NOT NULL,
    ComponentName VARCHAR(50) NOT NULL,
    Amount DECIMAL(10,2) NOT NULL CHECK (Amount >= 0),
    FOREIGN KEY (PayrollID) REFERENCES PayrollRun(PayrollID),
    FOREIGN KEY (EmpID) REFERENCES Employee(EmpID)
);
```

3. AttendanceRecord: -

Column Name	Data Type	Constraints
AttID (PK)	Integer	PRIMARY KEY
EmpID (FK)	Integer	NOT NULL
Date	DATE	NOT NULL
CheckIn	TIME	NOT NULL
CheckOut	TIME	NOT NULL
HoursWorked	DECIMAL(5,2)	CHECK
Status	VARCHAR/String	CHECK

```
CREATE TABLE AttendanceRecord ( AttID INT PRIMARY KEY,  
EmpID INT NOT NULL,  
Date DATE NOT NULL,  
CheckIn TIME NOT NULL,  
CheckOut TIME NOT NULL,  
HoursWorked DECIMAL(5,2)  
CHECK (HoursWorked >= 0),  
Status VARCHAR(20) CHECK (Status IN ('Present','Absent','Half-Day')),  
CONSTRAINT chk_att_time CHECK (CheckOut > CheckIn),  
FOREIGN KEY (EmpID) REFERENCES Employee(EmpID) );
```

4. ComplianceRecord : -

Column Name	Data Type	Constraints
ComplianceID (PK)	Integer	PRIMARY KEY
CompanyID (FK)	Integer	NOT NULL
ComplianceType	VARCHAR/String	NOT NULL
PeriodStart	DATE	NOT NULL
PeriodEnd	DATE	NOT NULL
SubmittedOn	DATE	NOT NULL

Status	VARCHAR/String	CHECK
Notes	VARCHAR/String	CHECK

```
CREATE TABLE ComplianceRecord ( ComplianceID INT PRIMARY KEY,
CompanyID INT NOT NULL,
ComplianceType VARCHAR(20) NOT NULL CHECK (ComplianceType IN ('PF','ESI','TDS')),
PeriodStart DATE NOT NULL,
PeriodEnd DATE NOT NULL,
SubmittedOn DATE NOT NULL,
Status VARCHAR(20) CHECK (Status IN ('Submitted','Pending')),
Notes VARCHAR(255),
CONSTRAINT chk_compliance_period CHECK (PeriodEnd >= PeriodStart),
FOREIGN KEY (CompanyID) REFERENCES Company(CompanyID) );
```

5. AttritionRecord: -

Column Name	Data Type	Constraints
AttritionID (PK)	Integer	PRIMARY KEY
EmpID (FK)	Integer	NOT NULL
ExitDate	DATE	DATE NOT NULL
Reason	VARCHAR/String	NOT NULL

```
CREATE TABLE AttritionRecord (
AttritionID INT PRIMARY KEY,
EmpID INT NOT NULL,
ExitDate DATE NOT NULL,
Reason VARCHAR(100) NOT NULL,
FOREIGN KEY (EmpID) REFERENCES Employee(EmpID)
);
```

6. ReportTemplate (Custom Report) : -

Column Name	Data Type	Constraints
TemplateID (PK)	Integer	PRIMARY KEY

CompanyID (FK)	Integer	NOT NULL
Name	VARCHAR/String	NOT NULL UNIQUE
Description	VARCHAR/String	NOT NULL
CreatedBy	Integer (FK)	NOT NULL
CreatedOn	DATE	NOT NULL
CriteriaJSON	JSON	FOREIGN KEY

```

CREATE TABLE ReportTemplate (
  TemplateID INT      PRIMARY KEY,
  CompanyID  INT      NOT NULL,
  Name       VARCHAR(100) NOT NULL UNIQUE,
  Description VARCHAR(255) NOT NULL,
  CreatedBy  INT      NOT NULL,
  CreatedOn  DATE      NOT NULL,
  CriteriaJSON JSON,
  FOREIGN KEY (CompanyID) REFERENCES Company(CompanyID),
  FOREIGN KEY (CreatedBy) REFERENCES SystemUser(SystemUserID)
);

```

7. ReportFilter :-

Column Name	Data Type	Constraints
FilterID (PK)	Integer	PRIMARY KEY
TemplateID (FK)	Integer	NOT NULL
FilterKey	VARCHAR/String	NOT NULL
FilterValue	VARCHAR/String	NOT NULL

```

CREATE TABLE ReportFilter ( FilterID INT PRIMARY KEY,
  TemplateID INT NOT NULL,
  FilterKey VARCHAR(50) NOT NULL,
  FilterValue VARCHAR(100) NOT NULL,
  FOREIGN KEY (TemplateID) REFERENCES ReportTemplate(TemplateID) );

```

8. ReportExportLog :-

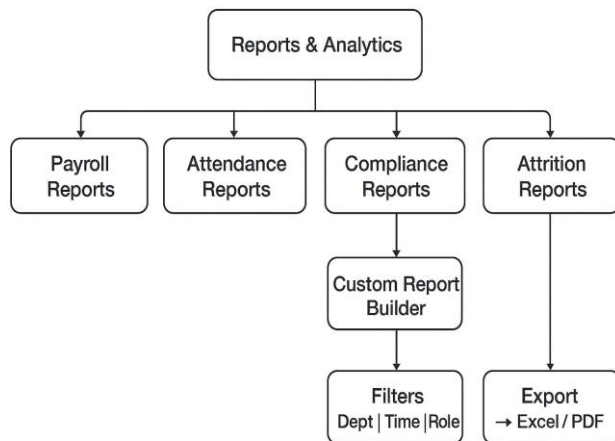
Column Name	Data Type	Constraints
ExportID (PK)	Integer	PRIMARY KEY
TemplateID (FK)	Integer	NOT NULL
ExportedBy (FK)	Integer	NOT NULL
ExportedOn	DATE	NOT NULL
Format	VARCHAR/String	CHECK
FilePath	VARCHAR/String	NOT NULL

```
CREATE TABLE ReportExportLog ( ExportID INT PRIMARY KEY,
TemplateID INT NOT NULL,
ExportedBy INT NOT NULL,
ExportedOn DATE NOT NULL,
Format VARCHAR(20) CHECK (Format IN ('PDF','Excel')),
FilePath VARCHAR(255) NOT NULL,
FOREIGN KEY (TemplateID) REFERENCES ReportTemplate(TemplateID),
FOREIGN KEY (ExportedBy) REFERENCES SystemUser(SystemUserID) );
```

ER Relationships

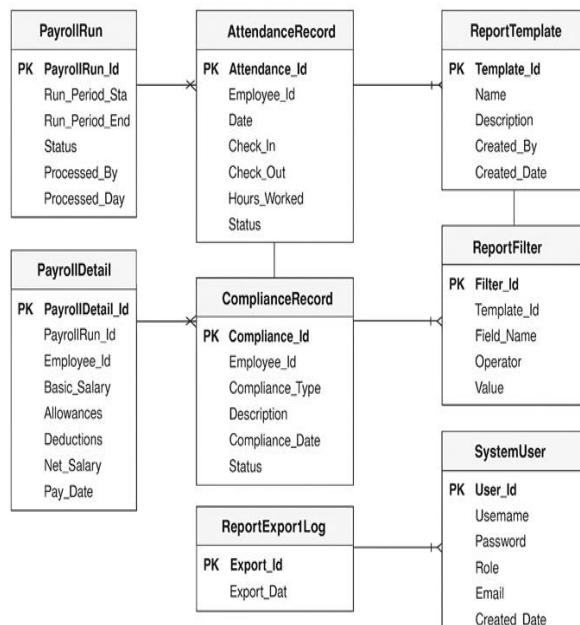
- **PayrollRun – Company:** Many payrolls runs belong to one company.
- **PayrollDetail – PayrollRun:** One payroll run can have multiple payroll details.
- **PayrollDetail – Employee:** One employee can have multiple payroll details.
- **AttendanceRecord – Employee:** One employee can have multiple attendance records.
- **ComplianceRecord – Company:** Many compliance records belong to one company.
- **AttritionRecord – Employee:** One employee can have multiple attrition records.
- **ReportTemplate – Company:** Many report templates belong to one company.
- **ReportTemplate – SystemUser (CreatedBy):** One system user can create multiple report templates.
- **ReportFilter – ReportTemplate:** One report template can have multiple filters.
- **ReportExportLog – ReportTemplate:** One report template can have multiple export logs.
- **ReportExportLog – SystemUser (ExportedBy):** One system user can export multiple reports.

Flow Diagram



Concept

ER Diagram:-



Module 1.9 Setting and Configuration Management

Key Entities and Attribute: (Suraj Sakarde)

1. User :-

Column Name	Data Type	Constraints
Id(pk)	Integer	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
employeeId(fk)	String	FOREIGN KEY REFERENCES Employee(EmployeeId)
userName	String	NOT NULL
email	String	NOT NULL
password	String	NOT NULL
roleId(fk)	Integer	FOREIGN KEY REFERENCES UserRole(roleId)
permissionId(fk)	integer	FOREIGN KEY REFERENCES RolePermission(PermissionId)

```
CREATE TABLE User (  
  Id INT PRIMARY KEY AUTO_INCREMENT,  
  employeeId VARCHAR(50) NOT NULL,  
  userName VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL,  
  roleId INT NOT NULL,  
  permissionId INT NOT NULL,  
  CONSTRAINT fk_user_employee FOREIGN KEY (employeeId) REFERENCES Employee(employeeId),  
  CONSTRAINT fk_user_role FOREIGN KEY (roleId) REFERENCES UserRole(roleId),  
  CONSTRAINT fk_user_permission FOREIGN KEY (permissionId) REFERENCES RolePermission(PermissionId)  
);
```

2. UserRole: -

Column Name	Data Type	Constraints
Id(pk)	Integer	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
name	String	NOT NULL
description	String	NOT NULL
userId(fk)	Integer	FOREIGN KEY REFERENCES User(userId)

```
CREATE TABLE UserRole (  
  Id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL,  
  description VARCHAR(255) NOT NULL,
```

```

userId INT NOT NULL,
CONSTRAINT fk_userrole_user FOREIGN KEY (userId) REFERENCES User(Id)
);

```

3. Role_Permission: -

Column Name	Data Type	Constraints
Id(pk)	Integer	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
name	String	NOT NULL
roleId(fk)	Integer	FOREIGN KEY REFERENCES UserRole(UserRoleId)
description	String	NOT NULL

```

CREATE TABLE RolePermission (
  Id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  roleId INT NOT NULL,
  description VARCHAR(255) NOT NULL,
  CONSTRAINT fk_permission_role FOREIGN KEY (roleId) REFERENCES UserRole(Id)
);

```

Entity Relationships

- **User↔UserRole** →One login application user has one userRole .
- **User↔Employee**→One user has one employee.
- **UserRole↔Role_Permission**→Many UserRoles has one permission.